# Rubric

Instructions: DELETE this cell before you submit via a `git push` to your repo before deadline. This cell is for your reference only and is not needed in your report.

Scoring: Out of 18 points in total. 15 points in the report. 3 points in the video summary of your report for a non-technical audience.

For the report:

- Each Developing => -0.5 pts
- Each Unsatisfactory/Missing => -1 pts
  - until the score is 0

For the video:

- Developing => -1.5 pts
- Unsatisfactory/Missing => -3 pts

| | Unsatisfactory | Developing | Proficient | Exellent |
|---|---|---|---|---|
| Title & abstract | Title or abstract is confusing or fails to offer important details about the question, variables, context, or methods of the project. | Title or abstract lacks relevance or fails to offer appropriate details about the question, variables, context, or methods of the project. | Title and abstract are relevant, offering sufficient details about the research project. | Title and abstract are informative, succinct, and clear. It offers specific details about the question, variables, context, and proposed methods of the study. |
| Background | Did not have at least 3 reliable and relevant sources. Or relevant sources were not used in relevant ways | A key component was not connected to the research literature. Selected literature was from unreliable sources. Literary supports were vague or ambiguous. | Key research components were connected to relevant, reliable literature. | Narrative integrates critical and logical details from the peer-reviewed or other professional research literature. Each key research component is grounded to the literature. Attention is given to different perspectives, threats to validity, and opinion vs. evidence. |

| | Unsatisfactory | Developing | Proficient | Exellent |
|---|---|---|---|---|
| Research question & hypothesis | Research issue remains unclear. The research purpose, questions, hypotheses, definitions or variables and controls are still largely undefined, or when they are are poorly formed, ambiguous, or not logically connected to the description of the problem. Unclear connections to the literature. | Research issue is identified, but statement is too broad or fails to establish the importance of the problem. The research purpose, questions, hypotheses, definitions or variables and controls are poorly formed, ambiguous, or not logically connected to the description of the problem. Unclear connections to the literature. | Identifies a relevant research issue. Research questions are succinctly stated, connected to the research issue, and supported by the literature. Variables and controls have been identified and described. Connections are established with the literature. | Presents a significant research problem related to the chemical sciences. Articulates clear, reasonable research questions given the purpose, design, and methods of the project. All variables and controls have been appropriately defined. Proposals are clearly supported from the research and theoretical literature. All elements are mutually supportive. |
| Data description | Did not have references to relevant data sources for this problem. Did not describe the data obtained at those sources | A key data source was not referenced or described in satisfactory level of detail | All relevant data sources were referenced and described in terms of their key variables and size | Multiple data sources for each aspect of the project, All data sources are fully described and referenced. The details of the descriptions also make it clear how they support the needs ot the project. |
| Data wrangling | Data obviously not tidy and not clean. Or even if they are clean/ tidy, there was no demonstration of that fact | Data has some minor problems with tidiness or cleanliness. Potentially these are not problems but it wasn't fully demonstrated to be the case in the code | Data are made tidy and clean, or are clearly demonstrated to be so from download | Data wrangling is complete, but also particularly thoroughly done for a very tough case. Often free text parsing is a bear like this, and doing it well can mean a million edge cases. |
| Data visualizations | Many plots don't capture useful information, or | Most plots make sense and capture useful | Plots make sense and capture useful | Plots are beautiful and information dense. Legends, |

| | Unsatisfactory | Developing | Proficient | Exellent |
|---|---|---|---|---|
| | their display of information is difficult to understand.. Legends, labels, axes, titles, captions, etc of many plots may be poor or missing. Figures are not referenced in the main text. | information. Legends, labels, axes, titles, captions, etc are mostly legible and adequately describe the content of the figure. Most figures are referenced or explained inside the main text and not just left hanging. | information. Legends, labels, axes, titles, captions, etc are clearly legible and adequately describe the content of the figure. Figures are referenced or explained inside the main text and not just left hanging. | labels, axes, titles, captions, etc clearly describe the content of the figure. Figures are well integrated with the text. |
| Data analysis and results | Lacks any metrics or at least metrics that are relevant to the problem | Some metrics selected are not appropriate for the problem or the metrics selected are not described sufficiently | All metrics described are appropriate and and described sufficiently | The metrics are described clearly and succinctly. Their appropriateness for addressing the research problem is clearly described. |
| Ethics | Only tries a baseline model, never advances to proper model/ hyper-parameter selection | Only investigates one minimal aspect of their problem, e.g. just did a single hyperparameter selection on a single model | Does a good model/hyper-parameter selection. Only attempts minimal secondary analysis | Does a good model/hyper-parameter selection. Performs multiple secondary analysis such as learning curves, heat maps looking at where in the parameter space things are good/ bad, uses statistical testing |
| Conclusion and discussion | Doesn't have a narrative/ conclusions of what the report accomplished. Or very minimal conclusions, like "X was best" without anything else | Has narrative/ conclusions, but it feels perfunctory and lacks context. Especially if the limitations and ethics sections are largely junk or missing | Properly recapitulates the results and provides context for them. Has a sense of the limitatins and ethical issues. | Clearly recapitulates the results and provides context, including the literature/ background Has a sense of what to do next, and has good explorations of the limitations and ethics. |

| | Unsatisfactory | Developing | Proficient | Exellent |
|---|---|---|---|---|
| Documentation and written communication | Code documentation is non-existent or perfunctory. Writing has usage errors, poor fluency, and poor rhetoric. If doc/writing are both like this you get unsatisfacory. If one is unsatisfactory and the other is developing you still get unsatisfactory overall. | Code documentation is only partially there. Writing has some usage errors, some poor fluency, or some poor rhetoric. If doc/writing are both like this you get developing. If one is developing and the other is satisfactory you still get developing. | Code is documented throughout. Writing has good fluency, good rhetoric, and at most a few minor errors. | Code documentation is highly informative with docstrings. Writing is high quality. If both doc/writing are like this you get excellent. If one is satisfactory and the other excellent you still get satisfactory. |
| Video (developing -1.5 points, unsatisfactory -3 points) | Video well outside the 3-5min time range. Does not communicate major results at a non-technical level. Has major technical issues such as no sound or not available at link. Unsatisfactory if 2/3 of these are true | Video just outside the 3-5min time range. Does not communicate well on some points (either wrong tech level or just confusing). Has technical issues such as poor speech quality on video. Developing if 2/3 of these are true | Video has good quality and is in within 3-5 min range. Communicates the major results at an accessible level for non-technical people. | Video has high production quality and is in within 3-5 min range. Communicates the major results clearly and gives a sense of the methods at an accessible level for non-technical people. Has a wow factor (e.g. funny or original approach) |
| Quality modifier (can result in extra credit beyond 18 points for this assignment) | Almost everything was just barely proficient. Lots of minimal effort work. Do not double punish; if they already lost points on the rubric do not invoke this: -0.5 points | | | More than 5 excellents on this rubric: +0.5 points |

# COGS 108 - Final Project (change this to your project's title)

# Permissions

Place an `X` in the appropriate bracket below to specify if you would like your group's project to be made available to the public, including the names of the authors.

- [ ] YES - make available
- [ ] NO - keep private

# Link to video

https://link.to.your.publicly.viewable.video

# Abstract

Please write one to four paragraphs that describe a very brief overview of why you did this, how you did, and the major findings and conclusions.

# Authors

**Aaron Chakma:** TLC Dataset & Data Wrangling, Drafting Overall Methodology, NY Maps (MTA Stations and Ride Hailing Zones), MTA Rider Histograms

**Amara Ihekwoeme:** Writing - Original, Writing - Review & Editing, Gas Price Dataset Wrangling, Gas Price Analysis

**Marco Krause:** Background Research, Review & Editing, and Conceptualization, TLC Dataset & Data Wrangling, TLC Rider Histograms

**Dustin Miller:** MTA Dataset & Data Wrangling, Writing - Revision, Project Lead, Rain and Snow Analysis

**Kera Yu:** Writing - Original, Writing - Review & Editing, Edit of Project Proposal, Daily Weather Dataset & Data Wrangling, Average Temperature Figures and Analysis

# Research Question

In New York City, how does taxi and ride hailing activity (Uber and Lyft proxy via TLC HVFHV trips) affect the nearest MTA subway station ridership (total riders per day) in 2023, controlling for trends such as seasonality, weather, and gas prices?

# Background and Prior Work

# Background

Public transit agencies have pursued service expansions and frequency improvements to attract riders, yet many U.S. cities experienced stagnation or decline in ridership in the years leading up to COVID-19. A Congressional Research Service review notes that despite major capital investments (including rail expansions), overall ridership fell in many large markets, and new rail lines do not necessarily increase total system ridership– sometimes shifting resources away from bus networks that carry a large share of trips.[1] At the same time, multiple competitive and behavioral forces changed the travel market: cheaper gasoline, rising fares, telework, and the growth of alternative modes like bikeshare and ride-hailing.[1]

# Prior work

Recent empirical work suggests that transit supply improvements can raise ridership, but the net outcome depends on the broader context. Erhardt et al. (2022) quantify ridership change across U.S. metropolitan areas pre-pandemic and find that while expanded transit service and land-use changes would have increased ridership, these gains were outweighed by other factors—especially ride-hailing along with higher fares, higher incomes, more teleworking, higher car ownership, and lower gas prices.[3] Complementing this, Graehler et al. (TRB 2019) document mode-specific ridership declines in major U.S. cities since the mid-2010s and enumerate commonly cited explanations including service cuts, deferred-maintenance reliability issues, micro mobility growth, and Transportation Network Company(TNC) expansion(Uber/Lyft); their panel analysis finds that standard factors (service levels, gas price, auto ownership) do not fully explain the recent declines and reports negative associations between years-since-TNC entry and bus/heavy-rail ridership.[2]

# Motivation for this project

New York City subway system is a critical part of its transportation with millions of people ridiing it daily. The increase in use of uber and lyft, with over half a million uber and lyft rides requested in New York City alone, people raise the question of whether these services pulling away riders from the subway system. It's a well known fact that public transport systems, such as MTA, generally lose money by being in operations. With ride hailing becoming a increasing way that people transport through the city, it may increase that defecit. To further investigate Erhardt et al. and Graehler et al. research claims, with our analysis we would be able to really pinpoint where the ridership declines and ridehailing increases.

# References (Footnotes)

1. Mallett, W. J. (2018). *Trends in Public Transportation Ridership: Implications for Federal*

*Policy* (CRS Report R45144). Congressional Research Service. https://www.congress.gov/crs_external_products/R/PDF/R45144/R45144.3.pdf ^

2. Erhardt, G. D., et al. (2022). Why has public transit ridership declined in the United States? *Transportation Research Part A*, 161, 68–87. https://www.sciencedirect.com/science/article/pii/S0965856422000945 ^

3. Tabassum, N., et al. (2025). Ways of increasing transit ridership—lessons learned from successful transit agencies. *Case Studies on Transport Policy*, 19. https://www.sciencedirect.com/science/article/pii/S2213624X24002177 ^

# Hypothesis

After accounting for trends (seasonality, weather, and gas prices), stations with high surrounding ride hailing activity will have a lower number of riders per day. We anticipate that during more intense weather conditions (heat, cold, rain, and snow), subway stations will experience less ridership and ride hailing activity will increase accordingly. Furthermore, we anticipate as gas prices increase, ridesharing prices will follow suit, lowering their ridership, and increasing subway ridership.

The hypothesized thresholds are as follows:

- Temperature (on days when it is not raining or snowing):
  - Lower threshold: 50°F (10°C) (temperatures below this will lead to a decrease in subway ridership)
  - Upper threshold: 80°F (27°C) (temperatures above this will lead to a decrease in subway ridership)
- Rain: any amount of rain will lead to a decrease in ridership
- Snow: any amount of snow will lead to a decrease in ridership
- Gas Prices: $3.50 (gas prices exceeding this amount will lead to increased subway ridership)

# Data

## Data overview

### MTA Subway Hourly Ridership (2023)

- Dataset Name: MTA Subway Hourly Ridership: 2020-2024
- Link to the dataset: ([https://data.ny.gov/Transportation/MTA-Subway-Hourly-Ridership-2020-2024/wujg-7c2s/about_data])
- Number of observations: 121 Million
- Number of variables: 12

- Description of the variables most relevant to this project:
  - transit_timestamp - The date and time in which the payment took place, rounded to the nearest hour
  - transit_mode - Distinguishes between the subway, Staten Island Railway, and the Roosevelt Island Tram. We will be using only the subway
  - station_complex - The subway complex where an entry swipe or tap took place. Large subway complexes, such as Times Square and Fulton Center, may contain multiple subway lines. The subway complex name includes the routes that stop at the complex in parenthesis, such as Zerega Av (6).
  - ridership - Total number of riders that entered a subway complex via OMNY or MetroCard at the specific hour and for that specific fare type.
  - latitude - Latitude for the specified subway complex
  - longitude - Longitude for the specified subway complex
- Descriptions of any shortcomings this dataset has with respect to the project:
  - The dataset is massive and will take some clever querying and processing to make it manageable.

### Ride Hailing Dataset

- Dataset Name: High Value For Hire Vehicle Data
- Link to the dataset: ([https://data.cityofnewyork.us/Transportation/2023-High-Volume-FHV-Trip-Data/u253-aew4/about_data])
- Number of observations: 707211
- Number of variables: 14
- Description of the variables most relevant to this project:
  - pickup_datetime - Pickup date and time (M/D/Y H:M:S)
  - PULocationID - Pickup location taxi zone ID
  - base_passenger_fare - base passenger fare before tolls, tips, taxes, and fees
  - tolls - total amount of tolls paid in trip
  - bcf - total amount collected in trip for black car fund
  - sales_tax - total amount collected in trip for NYS sales tax
  - congestion_surcharge - total amount collected in trip for NYS congestion surcharge
  - airport_fee -$2.50 for both drop off and pick up at LaGuardia, Newark, and John F. Kennedy airports
  - tips - total amount of tips received from passenger
- Descriptions of any shortcomings this dataset has with respect to the project:
  - This dataset's biggest shortcomings is its overachieving. It's quite literally every ride in New York on this day, so we need to compress the data greatly.

### Weather Dataset

- Dataset Name: NOAA NYC Daily Weather Summaries
- Link to the dataset: ([https://www.ncdc.noaa.gov/cdo-web/datasets/GHCND/locations/

CITY:US360019/detail])

- Number of observations: 33284
- Number of variables: 23
- Description of the variables most relevant to this project:
  - NAME - The name and abbreviated address of the weather station
  - DATE - Date in which an observation was made
  - PRCP (Precipitation) - Floating point value of the quantity of rain in inches
  - SNOW - Floating point value of the quantity of snowfall in inches
  - SNWD - Floating point value of the quantity of snow depth in inches
  - TAVG - Average temperature in degrees fahrenheit
  - TMAX - Maximum temperature in degrees fahrenheit
  - TMIN - Minimum temperature in degrees fahrenheit
- Descriptions of any shortcomings this dataset has with respect to the project:
  - Not all stations have all of the necessary weather variables (TMAX, TMIN, TAVG, PRCP, SNOW).
  - Some stations are missing a few days (~10-12) days of the year.

### Gas Price Dataset

- Dataset Name:Weekly New York City Regular All Formulations Retail Gasoline Prices
- Link to the dataset: ([https://www.eia.gov/dnav/pet/hist/LeafHandler.ashx?n=PET&s=EMM_EPMR_PTE_Y35NY_DPG&f=W])
- Number of observations: 1346
- Number of variables: 2
- Description of the variables most relevant to this project:
  - Date - The date that the gas price was recorded
  - Weekly New York City Regular All Formulations Retail Gasoline Prices (Dollars per Gallon) - Prices of regular gas in New York
- Descriptions of any shortcomings this dataset has with respect to the project:
  - Data is weekly, not daily.
  - City wide average of gas price, gas prices could vary by neighborhood.

**DELETE THE NOTE BEFORE SUBMISSION NOTE** The final report should not be overly long. If this wrangling step is huge and complicated you should remove these sections from the final report and replace them with a statement like "To see the code responsible for wrangling the data see `01-DataCheckpoint.ipynb` . From here on we will simply load the processed data for further analysis"

## MTA Subway Hourly Ridership (2023)

The main variable that we are interested in is ridership, which is the total number of riders that entered a subway recorded through the New York City payment system card. This ridership value includes transfers, those who use a free bus-to-subway or free out-of-

network transportation. For example, Time Square station complexes could get thousands of entries while outskirts complexes could get significantly less. The MTA Subway Hourly Ridership would allow us to see the times of peak ridership along with the lows of ridership.

A crucial variable that this dataset provides is transit_timestamps that are recorded to each hour, so a 1:37pm tie would be recorded as a 1:00pm transite_timestamp. The format is YYYY-MM-DDTHH:MM:SS.SSS, however the ":MM:SS.SSS" is always :00:00.000. As these are strings, this format allows us to use pd.to_datetime(), a more usable form. The location provided in the dataset is critical when analyzing other factors in comparison. Station_complex_id are the unique identifiers for station complexes while station_complex are the more human recognizable names of stations and these stations can support many subway lines. Another location specifying variable is latitude, longitude, and borough which also be critical for when compared to other factors.

One of the biggest concerns we have is the massive size of the datasets that we will be handling. To manage this we chose to use the "Query data" action on the website to reduce the dataset to the year 2023 and the specific variables we need. This resulted in a 3 GB CSV file which we processed locally to further reduce the size before adding the dataset to our Github repo.

Another concern is with ridership counts. Due to the ridership counts being counted based on fare payment transactions, those who evade subway fare will not be counted. So systemically, the ridership number will be underreported. MTA reports that subway fare evasion is around 10% which has decreased from the year before (https://cbcny.org/research/no-fare). Due to the volatile rate of fare evasion, it may bias the trend.

## Note:

The following ingestion and data wrangling is to demonstrate the procedure and is done on a 10,000 row subset of the full 2023 MTA dataset. The final output of the MTA dataset we are using in our project is "MTA_Subway_Daily_Manhattan.csv".

# Setup

This notebook should be run from the project root directory: `Group063_WI26` .

## 1. Create a virtual environment

We recommend creating a fresh virtual environment before installing dependencies.

Using `venv` :

```
python3 -m venv .venv
source .venv/bin/activate # On Windows, use: .venv\Scripts\activate
```

or in Windows

```
python3 -m venv .venv
.venv/Scripts/activate
```

## 2. Install dependencies

```
pip install -r requirements.txt
```

## 3. Select the correct Python kernel in Jupyter

In [2]:
```
from modules.run_imports import setup_final_project_imports

setup_final_project_imports(globals())
```

Out[2]:
```
{'datetime': datetime.datetime,
 'Point': shapely.geometry.point.Point,
 'box': <function shapely.geometry.geo.box(minx, miny, maxx, maxy, ccw=True)>,
 'gpd': <module 'geopandas' from 'c:\\Users\\theac\\OneDrive\\Documents\\GitHub\\G
roup063_WI26\\.venv\\Lib\\site-packages\\geopandas\\__init__.py'>,
 'graph': <module 'modules.graph' from 'c:\\Users\\theac\\OneDrive\\Documents\\Git
Hub\\Group063_WI26\\modules\\graph.py'>,
 'lowess': <function statsmodels.nonparametric.smoothers_lowess.lowess(endog, exog
, frac=0.6666666666666666, it=3, delta=0.0, xvals=None, is_sorted=False, missing='
drop', return_sorted=True)>,
 'maps': <module 'modules.maps' from 'c:\\Users\\theac\\OneDrive\\Documents\\GitHu
b\\Group063_WI26\\modules\\maps.py'>,
 'np': <module 'numpy' from 'c:\\Users\\theac\\OneDrive\\Documents\\GitHub\\Group0
63_WI26\\.venv\\Lib\\site-packages\\numpy\\__init__.py'>,
 'pd': <module 'pandas' from 'c:\\Users\\theac\\OneDrive\\Documents\\GitHub\\Group
063_WI26\\.venv\\Lib\\site-packages\\pandas\\__init__.py'>,
 'plt': <module 'matplotlib.pyplot' from 'c:\\Users\\theac\\OneDrive\\Documents\\G
itHub\\Group063_WI26\\.venv\\Lib\\site-packages\\matplotlib\\pyplot.py'>,
 'sns': <module 'seaborn' from 'c:\\Users\\theac\\OneDrive\\Documents\\GitHub\\Gro
up063_WI26\\.venv\\Lib\\site-packages\\seaborn\\__init__.py'>,
 'unary_union': <bound method CollectionOperator.unary_union of <shapely.ops.Colle
ctionOperator object at 0x000001DB5F21A3C0>>,
 'utils': <module 'modules.utils' from 'c:\\Users\\theac\\OneDrive\\Documents\\Git
Hub\\Group063_WI26\\modules\\utils.py'>,
 'subway_stations_gdf':      Station ID  Complex ID              Line
Stop Name  \
 0           317         611    Broadway - 7Av                  Times Sq-42 St
 1           515         515     Staten Island                       Annadale
 2           139         627  Franklin Shuttle                     Franklin Av
 3           349         349       Eastern Pky                       Junius St
 4           108         108         Myrtle Av  Middle Village-Metropolitan Av
 ..          ...         ...               ...                             ...
 491         222         222           63rd St                Roosevelt Island
 492          83          83           Jamaica                  Woodhaven Blvd
 493         514         514     Staten Island                     Eltingville
 494         232         232   6th Av - Culver                            2 Av
 495         348         348       Eastern Pky                     Rockaway Av

     Borough  GTFS Latitude  GTFS Longitude                    geometry
 0         M      40.755290      -73.987495   POINT (-73.9875 40.75529)
 1        SI      40.540460      -74.178217  POINT (-74.17822 40.54046)
 2        Bk      40.680596      -73.955827   POINT (-73.95583 40.6806)
 3        Bk      40.663515      -73.902447  POINT (-73.90245 40.66352)
 4         Q      40.711396      -73.889601    POINT (-73.8896 40.7114)
 ..      ...            ...             ...                         ...
 491       M      40.759145      -73.953260  POINT (-73.95326 40.75914)
 492       Q      40.693879      -73.851576  POINT (-73.85158 40.69388)
 493      SI      40.544601      -74.164570   POINT (-74.16457 40.5446)
 494       M      40.723402      -73.989938   POINT (-73.98994 40.7234)
 495      Bk      40.662549      -73.908946  POINT (-73.90895 40.66255)

 [496 rows x 8 columns],
 'taxi_zones_gdf':                     :id           :version
:created_at  \
 0    row-rgxy_42jk_dq7g  rv-a7vs.vwuy~tyci 2025-06-09 17:12:22.062000+00:00
 1    row-76qp.92qd~k3iu  rv-2qzz~i35d~7f8b 2025-06-09 17:12:22.062000+00:00
```

```
2    row-j55v~b782-bey7  rv-65ty-i8fr~zrr5 2025-06-09 17:12:22.062000+00:00
3    row-pnpa-ap69~6m7d  rv-wvih~xy8h~tpys 2025-06-09 17:12:22.062000+00:00
4    row-ewiw.9man~jjqn  rv-t95v.ubu4-6y4b 2025-06-09 17:12:22.062000+00:00
..                  ...                ...                            ...
258  row-6s67.hkk2-nsnx  rv-j8cy~mqwa~ux2g 2025-06-09 17:12:22.062000+00:00
259  row-9yp8~w8sp~n3vu  rv-62nh-xet5-u3jj 2025-06-09 17:12:22.062000+00:00
260  row-hfkk-5uf4~cgwj  rv-5y9s.dxvv.k5wv 2025-06-09 17:12:22.062000+00:00
261  row-48y3~fc59~vuhn  rv-5ftf_86ae_bzz8 2025-06-09 17:12:22.062000+00:00
262  row-w2eq~9th7_shem  rv-m2eu.7din_6es9 2025-06-09 17:12:22.062000+00:00

                         :updated_at       shape_leng          shape_area  \
0   2025-06-09 17:12:22.062000+00:00   0.116357453189     0.0007823067885
1   2025-06-09 17:12:22.062000+00:00    0.43346966679    0.00486634037837
2   2025-06-09 17:12:22.062000+00:00  0.0843411059012   0.000314414156821
3   2025-06-09 17:12:22.062000+00:00  0.0435665270921   0.000111871946192
4   2025-06-09 17:12:22.062000+00:00  0.0921464898574   0.000497957489363
..                               ...              ...                 ...
258 2025-06-09 17:12:22.062000+00:00   0.126750305191   0.000394552487366
259 2025-06-09 17:12:22.062000+00:00   0.133514154636   0.000422345326907
260 2025-06-09 17:12:22.062000+00:00  0.0271204563616  0.0000343423231652
261 2025-06-09 17:12:22.062000+00:00  0.0490636231541   0.000122330270966
262 2025-06-09 17:12:22.062000+00:00  0.0370166252994  0.0000657697664169

                       zone locationid        borough  \
0             Newark Airport          1            EWR
1                Jamaica Bay          2         Queens
2    Allerton/Pelham Gardens          3          Bronx
3              Alphabet City          4      Manhattan
4              Arden Heights          5  Staten Island
..                       ...        ...            ...
258       Woodlawn/Wakefield        259          Bronx
259                  Woodside        260         Queens
260       World Trade Center        261      Manhattan
261           Yorkville East        262      Manhattan
262           Yorkville West        263      Manhattan

                                            geometry
0    MULTIPOLYGON (((-74.18445 40.695, -74.18449 40...
1    MULTIPOLYGON (((-73.82338 40.63899, -73.82277 ...
2    MULTIPOLYGON (((-73.84793 40.87134, -73.84725 ...
3    MULTIPOLYGON (((-73.97177 40.72582, -73.97179 ...
4    MULTIPOLYGON (((-74.17422 40.56257, -74.17349 ...
..                                                ...
258  MULTIPOLYGON (((-73.85107 40.91037, -73.85207 ...
259  MULTIPOLYGON (((-73.90175 40.76078, -73.90147 ...
260  MULTIPOLYGON (((-74.01333 40.70503, -74.01327 ...
261  MULTIPOLYGON (((-73.94383 40.78286, -73.94376 ...
262  MULTIPOLYGON (((-73.95219 40.77302, -73.95269 ...

[263 rows x 10 columns],
'subway_lines_gdf':                    :id           :version
:created_at  \
0   row-f5yz~kdf9_eizy  rv-m6ve-y8pu-rhnq 2025-12-08 20:37:16.737000+00:00
1   row-36ii~ceiy.r87x  rv-298u-dmqi_945d 2025-12-08 20:37:16.737000+00:00
2   row-gft9_iv6d.ig7d  rv-dey5_q5et-aj24 2025-12-08 20:37:16.737000+00:00
3   row-igf9~5py8-yjpb  rv-jj4y_krfe-5nbd 2025-12-08 20:37:16.737000+00:00
```

```
4   row-xsdt-bugw.v77m  rv-32f4~bpk4-hgrh 2025-12-08 20:37:16.737000+00:00
5   row-ke66_tftq-h9cg  rv-6bdj_mci7~m4j6 2025-12-08 20:37:16.737000+00:00
6   row-j6qe~u395-2e5x  rv-27ha-kpe4_hshr 2025-12-08 20:37:16.737000+00:00
7   row-9f6j.6rpt_t94n  rv-isak.yz4g_cvgq 2025-12-08 20:37:16.737000+00:00
8   row-7ma7_3zn2.w2sa  rv-3dz7~ddjs_k9b8 2025-12-08 20:37:16.737000+00:00
9   row-ntu9_7hnw.75gy  rv-detc.9hhq-cgad 2025-12-08 20:37:16.737000+00:00
10  row-ykem-k6yb~ybtq  rv-z23r-axzj-95hh 2025-12-08 20:37:16.737000+00:00
11  row-fkkv~y3ir.y5ns  rv-3eyw~seh5-x87y 2025-12-08 20:37:16.737000+00:00
12  row-hnrh.b5v6~krkk  rv-wt97_aegc-5dqk 2025-12-08 20:37:16.737000+00:00
13  row-czxx~qib7~h7gf  rv-hz5r~875w-4eyq 2025-12-08 20:37:16.737000+00:00
14  row-akm7_jwyk.k8w3  rv-264x~jkic-i6u6 2025-12-08 20:37:16.737000+00:00
15  row-bmh8.sq65~t4zg  rv-q8cn.j8sd_bmdj 2025-12-08 20:37:16.737000+00:00
16  row-tbp8_4gdk~6asn  rv-t56m~upca-ijid 2025-12-08 20:37:16.737000+00:00
17  row-va6e~p666-dw6c  rv-w29t_gyy3.hf28 2025-12-08 20:37:16.737000+00:00
18  row-jitm-zeb9.zniu  rv-4r9n_f94y_jka9 2025-12-08 20:37:16.737000+00:00
19  row-k6cw-v4ne.8mpf  rv-q6kg~2fu2~9hb4 2025-12-08 20:37:16.737000+00:00
20  row-45vt-tkqd-x9rn  rv-dxch.xj4v~fhcm 2025-12-08 20:37:16.737000+00:00
21  row-zwh5~tw7i_dr9i  rv-g9a2.ss7t_sna2 2025-12-08 20:37:16.737000+00:00
22  row-qzck~hte7~mugb  rv-n9s8~jefs-f44r 2025-12-08 20:37:16.737000+00:00
23  row-rq8c~whv7-42vu  rv-5txf~pttn.ex65 2025-12-08 20:37:16.737000+00:00
24  row-vkh4~pwxw.7zba  rv-tjt2_qzkh.bda7 2025-12-08 20:37:16.737000+00:00
25  row-nig5-9pwm-fquy  rv-nmn2.at2t.48eu 2025-12-08 20:37:16.737000+00:00
26  row-ci9w-i8ji_4izu  rv-6xc6_9vfq_rw45 2025-12-08 20:37:16.737000+00:00
27  row-8kpf-95um.3jun  rv-54kc-2is5.hitk 2025-12-08 20:37:16.737000+00:00
28  row-hwbj.w7yb_kebd  rv-8p46.wqfh.ukag 2025-12-08 20:37:16.737000+00:00

                        :updated_at objectid                     service_name  \
0  2025-12-08 20:37:16.737000+00:00        5             Broadway / 7AV Local
1  2025-12-08 20:37:16.737000+00:00        4                 8 Avenue Express
2  2025-12-08 20:37:16.737000+00:00        3             14 St-Canarsie Local
3  2025-12-08 20:37:16.737000+00:00        2          Franklin Avenue Shuttle
4  2025-12-08 20:37:16.737000+00:00        1            Nassau Street Express
5  2025-12-08 20:37:16.737000+00:00       11               Lexington Av Local
6  2025-12-08 20:37:16.737000+00:00       10                42 Street Shuttle
7  2025-12-08 20:37:16.737000+00:00        9                 6 Avenue Express
8  2025-12-08 20:37:16.737000+00:00        8             Lexington Av Express
9  2025-12-08 20:37:16.737000+00:00        7                 Broadway Express
10 2025-12-08 20:37:16.737000+00:00        6                   Flushing Local
11 2025-12-08 20:37:16.737000+00:00       17                   6 Avenue Local
12 2025-12-08 20:37:16.737000+00:00       16  Brooklyn-Queens Crosstown Local
13 2025-12-08 20:37:16.737000+00:00       15                   6 Avenue Local
14 2025-12-08 20:37:16.737000+00:00       14            Nassau Street Express
15 2025-12-08 20:37:16.737000+00:00       13             Lexington Av Express
16 2025-12-08 20:37:16.737000+00:00       12             Lexington Av Express
17 2025-12-08 20:37:16.737000+00:00       22            Rockaway Park Shuttle
18 2025-12-08 20:37:16.737000+00:00       21                   8 Avenue Local
19 2025-12-08 20:37:16.737000+00:00       20                   8 Avenue Local
20 2025-12-08 20:37:16.737000+00:00       19                   Broadway Local
21 2025-12-08 20:37:16.737000+00:00       18                   Broadway Local
22 2025-12-08 20:37:16.737000+00:00       27            Staten Island Railway
23 2025-12-08 20:37:16.737000+00:00       26                 7 Avenue Express
24 2025-12-08 20:37:16.737000+00:00       25                 7 Avenue Express
25 2025-12-08 20:37:16.737000+00:00       24                 Broadway Express
26 2025-12-08 20:37:16.737000+00:00       23                 6 Avenue Express
27 2025-12-08 20:37:16.737000+00:00       31                   8 Avenue Local
28 2025-12-08 20:37:16.737000+00:00       30                 8 Avenue Express
```

```
   service      shape_stlength  \
0        1   77623.93076359815
1        A  101293.08009556108
2        L   54130.28254204164
3       SF    7098.21908922379
4        J   70656.68490562918
5        6   79571.48821306424
6       ST   2285.504908819597
7        B  127602.11278784153
8        4  106032.53413854787
9        Q   96999.90734425026
10       7   57050.04858324696
11       F  144263.22419393904
12       G  55554.243543589524
13       M   97721.04830678036
14       Z   70651.59442793574
15  5 Peak  22001.396947661535
16       5  128238.95785363218
17      SR  15617.842059676923
18       E   82658.30457308324
19       C   92661.23904982202
20       W   51871.98601416921
21       R  114757.83896293229
22     SIR  177179.50894550324
23       3   96290.94728194823
24       2  134418.98968148453
25       N  107239.95093678469
26       D   136463.1466545623
27       C   6049.811270225977
28       A   83974.93241524589

                                                 geometry
0   MULTILINESTRING ((-73.98594 40.75754, -73.9859...
1   MULTILINESTRING ((-73.94368 40.82556, -73.9436...
2   MULTILINESTRING ((-74.00342 40.74013, -74.0033...
3   MULTILINESTRING ((-73.95576 40.68104, -73.9557...
4   MULTILINESTRING ((-74.00382 40.71271, -74.0038...
5   MULTILINESTRING ((-74.00482 40.71245, -74.0047...
6   MULTILINESTRING ((-73.97919 40.75276, -73.9792...
7   MULTILINESTRING ((-73.96132 40.57754, -73.9605...
8   MULTILINESTRING ((-73.93294 40.66891, -73.9526...
9   MULTILINESTRING ((-73.98115 40.57743, -73.9810...
10  MULTILINESTRING ((-74.00612 40.75012, -74.0060...
11  MULTILINESTRING ((-73.98115 40.57743, -73.9809...
12  MULTILINESTRING ((-73.97941 40.64344, -73.9795...
13  MULTILINESTRING ((-73.88932 40.712, -73.88967 ...
14  MULTILINESTRING ((-74.00382 40.71271, -74.0038...
15  MULTILINESTRING ((-73.86958 40.84652, -73.8695...
16  MULTILINESTRING ((-73.94759 40.63219, -73.9475...
17  MULTILINESTRING ((-73.8163 40.60927, -73.81622...
18  MULTILINESTRING ((-73.81943 40.70721, -73.8193...
19  MULTILINESTRING ((-73.94368 40.82556, -73.9436...
20  MULTILINESTRING ((-74.00149 40.71982, -74.0012...
21  MULTILINESTRING ((-73.97902 40.68807, -73.9787...
22  MULTILINESTRING ((-74.25257 40.51227, -74.2515...
```

```
23  MULTILINESTRING ((-73.98594 40.75754, -73.9859...
24  MULTILINESTRING ((-73.98594 40.75754, -73.9859...
25  MULTILINESTRING ((-73.97902 40.68807, -73.9787...
26  MULTILINESTRING ((-73.97893 40.68816, -73.9789...
27  MULTILINESTRING ((-73.93934 40.84167, -73.9393...
28  MULTILINESTRING ((-73.80952 40.5933, -73.80951...  ,
'ridehail_df':              date  PULocationID  trip_count day_of_week  weekend
0      2023-01-01             1          58      Sunday     True
1      2023-01-01           171        1308      Sunday     True
2      2023-01-01           172         346      Sunday     True
3      2023-01-01           173        2218      Sunday     True
4      2023-01-01           174        2562      Sunday     True
...           ...           ...         ...         ...      ...
94940  2023-12-31            93         166      Sunday     True
94941  2023-12-31            94        1735      Sunday     True
94942  2023-12-31            95        4336      Sunday     True
94943  2023-12-31            97        4478      Sunday     True
94944  2023-12-31           265         146      Sunday     True

[94945 rows x 5 columns],
'mta_df':        station_complex_id        date  ridership day_of_week  weekend
0                        8  2023-01-01       8909      Sunday     True
1                        8  2023-01-03       9291     Tuesday    False
2                        8  2023-01-04      10697   Wednesday    False
3                        8  2023-01-05      10391    Thursday    False
4                        8  2023-01-06       8739      Friday    False
...                    ...         ...        ...         ...      ...
42196                  635  2023-12-27      18878   Wednesday    False
42197                  635  2023-12-28      20035    Thursday    False
42198                  635  2023-12-29      23840      Friday    False
42199                  635  2023-12-30      15912    Saturday     True
42200                  635  2023-12-31      11406      Sunday     True

[42201 rows x 5 columns]}
```

In [3]:

```
import pandas as pd
MTA_hrly_23 = pd.read_csv("data/00-raw/MTA_Subway_Hourly_Ridership__2020-2024_20260
print("The total number of missing values:", MTA_hrly_23.isna().sum().sum())
print('The only transit mode in our dataset is: ', MTA_hrly_23.transit_mode.unique(
# And, focusing specifically on the Manhattan borough...
mnhtn_MTA_hrly_23 = MTA_hrly_23.loc[MTA_hrly_23['borough'] == 'Manhattan'].copy()
print('The only borough in our dataset is: ', mnhtn_MTA_hrly_23.borough.unique())
# Therefore we can drop the borough and transit_mode columns.
mnhtn_MTA_hrly_23 = mnhtn_MTA_hrly_23.drop(['transit_mode','borough'], axis = 1)
# Next, we can create a map of the station_complex_ids and the station_complexes to
mnhtn_MTA_hrly_23['station_complex'] = mnhtn_MTA_hrly_23['station_complex'].str.str
mnhtn_MTA_hrly_23['Georeference'] = mnhtn_MTA_hrly_23['Georeference'].str.strip()
st_cmplx_unique = (
    mnhtn_MTA_hrly_23
    .drop_duplicates(subset=['station_complex_id'], keep='first')
    .sort_values('station_complex_id')
    .set_index('station_complex_id')[['station_complex', 'Georeference']]
)
# Lastly, latitude and longitude are both contained within the Georeference column.
# Now we can drop the station_complex, latitude, longitude, and Georeference column
mnhtn_MTA_hrly_23 = mnhtn_MTA_hrly_23.drop(['station_complex','latitude','longitude
```

```
mnhtn_MTA_hrly_23['transit_timestamp'] = pd.to_datetime(mnhtn_MTA_hrly_23['transit_
mnhtn_MTA_hrly_23['date'] = mnhtn_MTA_hrly_23['transit_timestamp'].dt.date
mnhtn_MTA_hrly_23['ridership'] = pd.to_numeric(mnhtn_MTA_hrly_23['ridership'].astyp
daily_ridership = (mnhtn_MTA_hrly_23.groupby(['station_complex_id', 'date'])['rider
daily_ridership.to_csv("data/01-interim/MTA_Subway_Daily_Manhattan_DEMO.csv", index
st_cmplx_unique.to_csv("data/01-interim/MTA_Subway_Daily_Manhattan_Stations_DEMO.cs
```

```
The total number of missing values: 0
The only transit mode in our dataset is:  <ArrowStringArray>
['subway']
Length: 1, dtype: str
The only borough in our dataset is:  <ArrowStringArray>
['Manhattan']
Length: 1, dtype: str
```

In [4]:

```
mta_df = pd.read_csv("data/01-interim/MTA_subway/MTA_Subway_Daily_Manhattan.csv")

mta_totals_df = mta_df.groupby('date')['ridership'].sum().reset_index()
mta_df['date'] = pd.to_datetime(mta_df['date'])
print(mta_df.columns)
# Convert dates
mta_totals_df['date'] = pd.to_datetime(mta_totals_df['date'])

# Add day of week columns
mta_totals_df['day_of_week'] = mta_totals_df['date'].dt.day_name()

# Add weekend column (using dayofweek numbers: 5=Saturday, 6=Sunday)
mta_totals_df['weekend'] = mta_totals_df['date'].dt.dayofweek.isin([5, 6])

# Split data into weekday and weekend
weekday_data = mta_totals_df[~mta_totals_df['weekend']].copy()
weekend_data = mta_totals_df[mta_totals_df['weekend']].copy()

print(weekday_data.columns)

# Calculate statistics for weekdays
weekday_mean = weekday_data['ridership'].mean()
weekday_std = weekday_data['ridership'].std()
weekday_upper_threshold = weekday_mean + (2 * weekday_std)
weekday_lower_threshold = weekday_mean - (2 * weekday_std)
weekday_upper_outliers = weekday_data[weekday_data['ridership'] > weekday_upper_thr
weekday_lower_outliers = weekday_data[weekday_data['ridership'] < weekday_lower_thr

# Calculate statistics for weekends
weekend_mean = weekend_data['ridership'].mean()
weekend_std = weekend_data['ridership'].std()
weekend_upper_threshold = weekend_mean + (2 * weekend_std)
weekend_lower_threshold = weekend_mean - (2 * weekend_std)
weekend_upper_outliers = weekend_data[weekend_data['ridership'] > weekend_upper_thr
weekend_lower_outliers = weekend_data[weekend_data['ridership'] < weekend_lower_thr

# Print weekday outliers
print(f"\nWEEKDAY - HIGH OUTLIERS (above +2σ): {len(weekday_upper_outliers)} days f
print("-" * 50)
if len(weekday_upper_outliers) > 0:
    for idx, row in weekday_upper_outliers.iterrows():
```

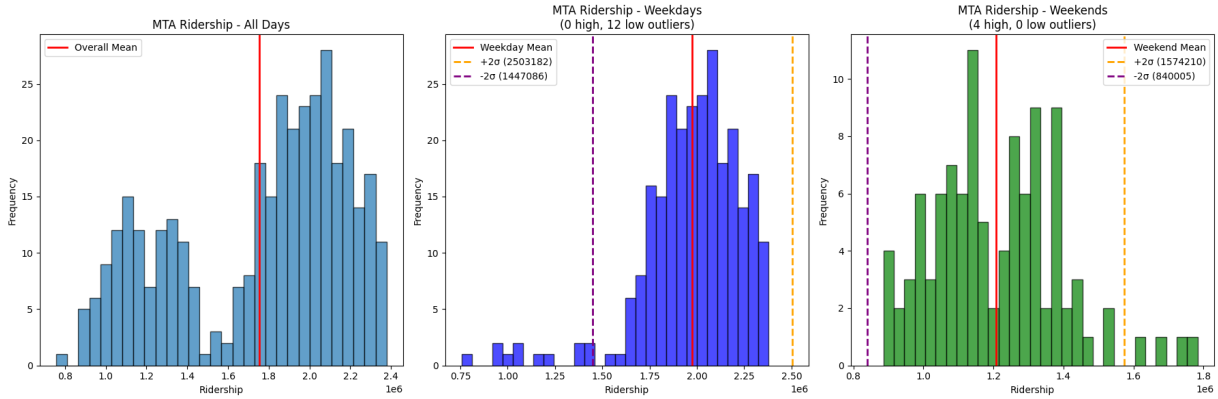
```
        print(f"Date: {row['date'].strftime('%Y-%m-%d')}, "
            f"Ridership: {row['ridership']:.0f}, "
            f"Day: {row['day_of_week']} "
            f"(+{(row['ridership'] - weekday_mean)/weekday_std:.2f}σ)")
else:
    print("No weekday high outliers found above +2σ")

print(f"\nWEEKDAY - LOW OUTLIERS (below -2σ): {len(weekday_lower_outliers)} days fo
print("-" * 50)
if len(weekday_lower_outliers) > 0:
    for idx, row in weekday_lower_outliers.iterrows():
        print(f"Date: {row['date'].strftime('%Y-%m-%d')}, "
            f"Ridership: {row['ridership']:.0f}, "
            f"Day: {row['day_of_week']} "
            f"({(row['ridership'] - weekday_mean)/weekday_std:.2f}σ)")
else:
    print("No weekday low outliers found below -2σ")

# Print weekend outliers
print(f"\nMean ridership (Weekends): {weekend_mean:.2f}")
print(f"Standard deviation: {weekend_std:.2f}")
print(f"Upper 2-sigma threshold (+2σ): {weekend_upper_threshold:.2f}")
print(f"Lower 2-sigma threshold (-2σ): {weekend_lower_threshold:.2f}")

print(f"\nWEEKEND - HIGH OUTLIERS (above +2σ): {len(weekend_upper_outliers)} days f
print("-" * 50)
if len(weekend_upper_outliers) > 0:
    for idx, row in weekend_upper_outliers.iterrows():
        print(f"Date: {row['date'].strftime('%Y-%m-%d')}, "
            f"Ridership: {row['ridership']:.0f}, "
            f"Day: {row['day_of_week']} "
            f"(+{(row['ridership'] - weekend_mean)/weekend_std:.2f}σ)")
else:
    print("No weekend high outliers found above +2σ")

print(f"\nWEEKEND - LOW OUTLIERS (below -2σ): {len(weekend_lower_outliers)} days fo
print("-" * 50)
if len(weekend_lower_outliers) > 0:
    for idx, row in weekend_lower_outliers.iterrows():
        print(f"Date: {row['date'].strftime('%Y-%m-%d')}, "
            f"Ridership: {row['ridership']:.0f}, "
            f"Day: {row['day_of_week']} "
            f"({(row['ridership'] - weekend_mean)/weekend_std:.2f}σ)")
else:
    print("No weekend low outliers found below -2σ")
```

```
Index(['station_complex_id', 'date', 'ridership'], dtype='str')
Index(['date', 'ridership', 'day_of_week', 'weekend'], dtype='str')

WEEKDAY - HIGH OUTLIERS (above +2σ): 0 days found
--------------------------------------------------
No weekday high outliers found above +2σ

WEEKDAY - LOW OUTLIERS (below -2σ): 12 days found
--------------------------------------------------
Date: 2023-01-02, Ridership: 1062726, Day: Monday (-3.46σ)
Date: 2023-01-16, Ridership: 1135997, Day: Monday (-3.18σ)
Date: 2023-02-20, Ridership: 1189966, Day: Monday (-2.97σ)
Date: 2023-05-29, Ridership: 1048880, Day: Monday (-3.51σ)
Date: 2023-06-19, Ridership: 1407937, Day: Monday (-2.15σ)
Date: 2023-07-03, Ridership: 1423957, Day: Monday (-2.09σ)
Date: 2023-07-04, Ridership: 952323, Day: Tuesday (-3.87σ)
Date: 2023-09-04, Ridership: 996208, Day: Monday (-3.71σ)
Date: 2023-09-29, Ridership: 1364610, Day: Friday (-2.31σ)
Date: 2023-11-23, Ridership: 941205, Day: Thursday (-3.92σ)
Date: 2023-11-24, Ridership: 1398884, Day: Friday (-2.18σ)
Date: 2023-12-25, Ridership: 757361, Day: Monday (-4.61σ)

Mean ridership (Weekends): 1207107.10
Standard deviation: 183551.21
Upper 2-sigma threshold (+2σ): 1574209.52
Lower 2-sigma threshold (-2σ): 840004.67

WEEKEND - HIGH OUTLIERS (above +2σ): 4 days found
--------------------------------------------------
Date: 2023-10-28, Ridership: 1606262, Day: Saturday (+2.17σ)
Date: 2023-12-02, Ridership: 1670874, Day: Saturday (+2.53σ)
Date: 2023-12-09, Ridership: 1783307, Day: Saturday (+3.14σ)
Date: 2023-12-16, Ridership: 1745770, Day: Saturday (+2.93σ)

WEEKEND - LOW OUTLIERS (below -2σ): 0 days found
--------------------------------------------------
No weekend low outliers found below -2σ
```

In [5]:

```
# Create histograms
# Set up the figure
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Histogram for all days
axes[0].hist(mta_totals_df['ridership'], bins=30, edgecolor='black', alpha=0.7)
axes[0].axvline(mta_totals_df['ridership'].mean(), color='red', linestyle='-', line
axes[0].set_title(f'MTA Ridership - All Days')
axes[0].set_xlabel('Ridership')
axes[0].set_ylabel('Frequency')
axes[0].legend()

# Histogram for weekdays with thresholds
axes[1].hist(weekday_data['ridership'], bins=30, edgecolor='black', alpha=0.7, colo
axes[1].axvline(weekday_mean, color='red', linestyle='-', linewidth=2, label='Weekd
axes[1].axvline(weekday_upper_threshold, color='orange', linestyle='--', linewidth=
                label=f'+2σ ({weekday_upper_threshold:.0f})')
axes[1].axvline(weekday_lower_threshold, color='purple', linestyle='--', linewidth=
```

```
                label=f'-2σ ({weekday_lower_threshold:.0f})')
axes[1].set_title(f'MTA Ridership - Weekdays\n({len(weekday_upper_outliers)} high,
axes[1].set_xlabel('Ridership')
axes[1].set_ylabel('Frequency')
axes[1].legend()

# Histogram for weekends with thresholds
axes[2].hist(weekend_data['ridership'], bins=30, edgecolor='black', alpha=0.7, colo
axes[2].axvline(weekend_mean, color='red', linestyle='-', linewidth=2, label='Weeke
axes[2].axvline(weekend_upper_threshold, color='orange', linestyle='--', linewidth=
                label=f'+2σ ({weekend_upper_threshold:.0f})')
axes[2].axvline(weekend_lower_threshold, color='purple', linestyle='--', linewidth=
                label=f'-2σ ({weekend_lower_threshold:.0f})')
axes[2].set_title(f'MTA Ridership - Weekends\n({len(weekend_upper_outliers)} high,
axes[2].set_xlabel('Ridership')
axes[2].set_ylabel('Frequency')
axes[2].legend()

plt.tight_layout()
plt.show()

# Identify outlier dates to remove
outlier_dates = set()

# Add weekday outlier dates
outlier_dates.update(weekday_upper_outliers['date'].tolist())
outlier_dates.update(weekday_lower_outliers['date'].tolist())

# Add weekend outlier dates
outlier_dates.update(weekend_upper_outliers['date'].tolist())
outlier_dates.update(weekend_lower_outliers['date'].tolist())

print(outlier_dates)

# Remove outlier dates from the original dataframe
mta_clean = mta_df[~mta_df['date'].isin(outlier_dates)].copy()
mta_clean['day_of_week'] = mta_clean['date'].dt.day_name()
mta_clean['weekend'] = mta_clean['date'].dt.dayofweek.isin([5, 6])

# Reset index for cleanliness
mta_clean = mta_clean.reset_index(drop=True)

# Save to CSV
mta_clean.to_csv('data/02-processed/MTA_Ridership_cleaned.csv', index=False)
```




```
{Timestamp('2023-01-02 00:00:00'), Timestamp('2023-01-16 00:00:00'), Timestamp('202
3-10-28 00:00:00'), Timestamp('2023-12-02 00:00:00'), Timestamp('2023-05-29 00:00:0
0'), Timestamp('2023-12-09 00:00:00'), Timestamp('2023-07-04 00:00:00'), Timestamp
('2023-06-19 00:00:00'), Timestamp('2023-12-16 00:00:00'), Timestamp('2023-02-20 0
0:00:00'), Timestamp('2023-11-24 00:00:00'), Timestamp('2023-09-04 00:00:00'), Times
tamp('2023-07-03 00:00:00'), Timestamp('2023-09-29 00:00:00'), Timestamp('2023-12-25
00:00:00'), Timestamp('2023-11-23 00:00:00')}
```

## Ride Hailing Dataset

Our dataset is provided by the NYC government, so it's publicly accessible, and contains transport data from Janurary 1st 2023 to Janurary 1st 2024 for high volume for hire vehicles. This includes services like Uber and Lyft. We chose this dataset because it aggregates all rideshare companies, and since their inception it takes up about 80% of the marketshare for ride-hailing services.

The dataset has lots of metrics, but the ones that we are going to apply to our project are the pickup date and time, the location ID (Taxi Zones), the distance traveled, the trip time, and a lot of the cost components like sales tax, base price, and additional fees. To clean the data, we aggregated the costs and combined the columns into a single total cost column. Our immediate goals don't necessitate these metrics, but we might use them for more analysis later on, so we placed it in the 01-interim folder temporarily. The dataset that we were provided was nearly perfect, and we did our due diligence checking for any abnormal behavior, but we concluded that it was fine as is, just needed to remove some columns.

Note that due to the rather large datasize, we were unable to upload it via the URL, so we manually uploaded the 80MB file ourselves. The cleaned interim data is about 30MB, but this is just for one day. To concatenate the data, I created a function that can aggregate the entries into rides per hour per each taxi zone. This now matches the hourly format of the MTA data, and is much more processable at 172KB. Converting each day to this format will prove challenging due to the limitations of the API, and the aforementioned large file sizes, but not impossible.

In [6]:
```
ridehail_df = pd.read_csv("data/01-interim/TLC_ridehail/ridehailing_daily_ridership
print(ridehail_df.columns)
# Convert dates
ridehail_df['date'] = pd.to_datetime(ridehail_df['date'])
```

```
# Add day of week columns
ridehail_df['day_of_week'] = ridehail_df['date'].dt.day_name()

# Add weekend column (using dayofweek numbers: 5=Saturday, 6=Sunday)
ridehail_df['weekend'] = ridehail_df['date'].dt.dayofweek.isin([5, 6])
ridehail_total = ridehail_df.groupby('date', as_index=False)['trip_count'].sum()

# Split data into weekday and weekend
weekday_data = ridehail_df[~ridehail_df['weekend']]
weekend_data = ridehail_df[ridehail_df['weekend']]

# Aggregate to daily totals
weekday_total = weekday_data.groupby('date', as_index=False)['trip_count'].sum()
weekend_total = weekend_data.groupby('date', as_index=False)['trip_count'].sum()

# Merge back day_of_week for printing purposes
weekday_total = weekday_total.merge(weekday_data[['date', 'day_of_week']].drop_dupl
weekend_total = weekend_total.merge(weekend_data[['date', 'day_of_week']].drop_dupl

# Calculate statistics for weekdays
weekday_mean = weekday_total['trip_count'].mean()
weekday_std = weekday_total['trip_count'].std()
weekday_upper_threshold = weekday_mean + (3 * weekday_std)
weekday_lower_threshold = weekday_mean - (3 * weekday_std)
weekday_upper_outliers = weekday_total[weekday_total['trip_count'] > weekday_upper_
weekday_lower_outliers = weekday_total[weekday_total['trip_count'] < weekday_lower_

# Calculate statistics for weekends
weekend_mean = weekend_total['trip_count'].mean()
weekend_std = weekend_total['trip_count'].std()
weekend_upper_threshold = weekend_mean + (3 * weekend_std)
weekend_lower_threshold = weekend_mean - (3 * weekend_std)
weekend_upper_outliers = weekend_total[weekend_total['trip_count'] > weekend_upper_
weekend_lower_outliers = weekend_total[weekend_total['trip_count'] < weekend_lower_

# Print weekday outliers
print(f"\nWEEKDAY - HIGH OUTLIERS (above +3σ): {len(weekday_upper_outliers)} days f
print("-" * 50)
if len(weekday_upper_outliers) > 0:
    for idx, row in weekday_upper_outliers.iterrows():
        print(f"Date: {row['date'].strftime('%Y-%m-%d')}, "
            f"trip_count: {row['trip_count']:.0f}, "
            f"Day: {row['day_of_week']} "
            f"(+{(row['trip_count'] - weekday_mean)/weekday_std:.2f}σ)")
else:
    print("No weekday high outliers found above +3σ")

print(f"\nWEEKDAY - LOW OUTLIERS (below -3σ): {len(weekday_lower_outliers)} days fo
print("-" * 50)
if len(weekday_lower_outliers) > 0:
    for idx, row in weekday_lower_outliers.iterrows():
        print(f"Date: {row['date'].strftime('%Y-%m-%d')}, "
            f"trip_count: {row['trip_count']:.0f}, "
            f"Day: {row['day_of_week']} "
            f"({(row['trip_count'] - weekday_mean)/weekday_std:.2f}σ)")
```

```
else:
    print("No weekday low outliers found below -3σ")

# Print weekend statistics
print(f"\nWeekend Statistics:")
print(f"Mean trip_count (Weekends): {weekend_mean:.2f}")
print(f"Standard deviation: {weekend_std:.2f}")
print(f"Upper 3-sigma threshold (+3σ): {weekend_upper_threshold:.2f}")
print(f"Lower 3-sigma threshold (-3σ): {weekend_lower_threshold:.2f}")

print(f"\nWEEKEND - HIGH OUTLIERS (above +3σ): {len(weekend_upper_outliers)} days f
print("-" * 50)
if len(weekend_upper_outliers) > 0:
    for idx, row in weekend_upper_outliers.iterrows():
        print(f"Date: {row['date'].strftime('%Y-%m-%d')}, "
            f"trip_count: {row['trip_count']:.0f}, "
            f"Day: {row['day_of_week']} "
            f"(+{(row['trip_count'] - weekend_mean)/weekend_std:.2f}σ)")
else:
    print("No weekend high outliers found above +3σ")

print(f"\nWEEKEND - LOW OUTLIERS (below -3σ): {len(weekend_lower_outliers)} days fo
print("-" * 50)
if len(weekend_lower_outliers) > 0:
    for idx, row in weekend_lower_outliers.iterrows():
        print(f"Date: {row['date'].strftime('%Y-%m-%d')}, "
            f"trip_count: {row['trip_count']:.0f}, "
            f"Day: {row['day_of_week']} "
            f"({(row['trip_count'] - weekend_mean)/weekend_std:.2f}σ)")
else:
    print("No weekend low outliers found below -3σ")
```

```
Index(['date', 'PULocationID', 'trip_count'], dtype='str')

WEEKDAY - HIGH OUTLIERS (above +3σ): 0 days found
--------------------------------------------------
No weekday high outliers found above +3σ

WEEKDAY - LOW OUTLIERS (below -3σ): 1 days found
--------------------------------------------------
Date: 2023-01-02, trip_count: 479217, Day: Monday (-3.00σ)

Weekend Statistics:
Mean trip_count (Weekends): 809889.60
Standard deviation: 86286.58
Upper 3-sigma threshold (+3σ): 1068749.35
Lower 3-sigma threshold (-3σ): 551029.85

WEEKEND - HIGH OUTLIERS (above +3σ): 0 days found
--------------------------------------------------
No weekend high outliers found above +3σ

WEEKEND - LOW OUTLIERS (below -3σ): 0 days found
--------------------------------------------------
No weekend low outliers found below -3σ
```

## Weather Dataset

This dataset is key for avoiding incorrect analysis with weather as a confounding variable. The important metrics are the max, min, and average temperature (in fahrenheit), precipitation (in inches of rain), and snow (in inches of snow). These are important as inconvenient weather may be the difference between walking to a subway station or paying a premium to hail a taxi or uber. Additionally, rain could decrease subway ridership through less desired conditions, such as humidity, dampness, or smell, while snow could also affect ridership in both modes of transportation.

Major concerns for this dataset are primarily around the amount of missing values or NaNs. Additionally some columns are of mixed data type and need to be cleaned.

In [7]:
```
#import pandas as pd
#import numpy as np
#load the dataset
df_w = pd.read_csv("data/00-raw/NYC2023Weather.csv")
#print size of data set
print('The size of weather data is:',len(df_w))
```

```
The size of weather data is: 33284
```

In [8]:
```
#missing data
print('Number of N/A data:', df_w.isna().sum(axis=1).sum())
print('Number of missing data per column:',df_w.isna().sum(axis=0))
```

```
Number of N/A data: 485512
Number of missing data per column: STATION          0
NAME             0
LATITUDE         0
LONGITUDE        0
ELEVATION        0
DATE             0
DAPR         32700
MDPR         32704
PRCP           779
SNOW         11522
SNWD         24336
TAVG         32189
TMAX         27747
TMIN         27745
TOBS         30817
WT01         32536
WT02         33225
WT03         33044
WT04         33256
WT05         33282
WT06         33284
WT08         33073
WT11         33273
dtype: int64
```

In [9]:
```
#most of missing data is in variables we do not need
```

```
#how many stations have all the data we need?
df_w_na = df_w.dropna(subset = ['STATION','NAME','LONGITUDE','LATITUDE','DATE','TMA
print('Number of valid stations:', len(df_w_na['NAME'].unique()), ', more than enou
```

```
Number of valid stations: 11 , more than enough!
```

In [10]:

```
## make the dataset tidy
#trim unnecessary data and rename columns
rename_map = {'STATION': 'Station', 'NAME': 'Name', 'LATITUDE': 'Latitude', 'LONGIT
col_keep = ['STATION', 'NAME', 'LATITUDE', 'LONGITUDE', 'DATE', 'PRCP', 'SNOW', 'TA

df_w = df_w[col_keep].rename(columns = rename_map)
df_w.head()

df_w = df_w.dropna(subset = ['Station','Name','Latitude','Longitude','Date','Max_Te

#categorize snow and rainy days in new column (boolean)
df_w['Rain?'] = np.where(df_w['Precipitation'] > 0, 'yes', 'no')
df_w['Snow?'] = np.where(df_w['Snow'] > 0, 'yes', 'no')

#indexing fix
df_w = df_w.reset_index(drop = True)

#some days are missing average values, let's assume the mean of Max_Temp and Min_Te
temp_means = (df_w['Max_Temp'] + df_w['Min_Temp']) / 2
df_w['Avg_Temp'] = df_w['Avg_Temp'].fillna(temp_means)

#make sure all dates are in proper format and convert to date time
df_w['Date'] = pd.to_datetime(df_w['Date'])

# split dataset by state
df_ny_w = df_w[df_w['Name'].str.contains('NY')]
df_nj_w = df_w[df_w['Name'].str.contains('NJ')]

#narrow down to manhattan, using Central Park station data
df_central_park = df_ny_w[df_ny_w['Name'] == 'NY CITY CENTRAL PARK, NY US']
#another indexing fix
df_central_park = df_central_park.reset_index(drop = True)
```

In [11]:

```
#check for inconsistencies (unhash for more info)
#df_central_park[['Precipitation','Snow', 'Avg_Temp', 'Max_Temp', 'Min_Temp']].desc
print('Minimum recorded temperature:',df_central_park['Min_Temp'].min(), 'Farenheig
print('Day of minimum temperature (index, date):', df_central_park['Date'].loc[df_c
print('According to Google, 2/4/23-2/5/23 had extreme lows of around 3°F. May be a
```

```
Minimum recorded temperature: 3.0 Farenheight
Day of minimum temperature (index, date): 34   2023-02-04
Name: Date, dtype: datetime64[us]
According to Google, 2/4/23-2/5/23 had extreme lows of around 3°F. May be a potentia
l future outlier.
```

In [12]:

```
#save to data/02-processed (unhash to save)
#df_central_park.to_csv('data/02-processed/CLEANED_CentralPark_Weather2023.csv', in
#df_ny_w.to_csv('data/02-processed/CLEANED_NewYork_Weather2023.csv', index = False)
#df_nj_w.to_csv('data/02-processed/CLEANED_NewJersey_Weather2023.csv', index = Fals
```

## Gas Price Dataset

The organization that published this dataset is the U.S. Energy Information Administration (EIA) which is an independent federal statistical agency that collects, analyzes, and disseminates independent and impartial energy information. This specific dataset contains weekly average retail prices for regular gasoline in the New York City area in dollars per gallon. The dataset has two variables: the week-ending date and the average retail price in USD per gallon. It covers data from 2000 to present year which means there are approximately 1,340 weekly observations. The prices that consumers pay range from a little more than a dollar to nearly five dollars per gallon.

The data reveals major price spikes that coincide with significant events, such as Hurricane Katrina, which hit oil-producing regions and caused a spike in prices. In 2008, oil prices were at an all-time high that could be attributed to financial speculation and coinciding with the US being amidst a great recession. The peak of 2022 can be suggested to be due to the post pandemic activities that occur after things start opening up after a shutdown. Overall, lower prices under 3.00 dollars per gallon tend to correspond with periods of economic downs while prices above 4.00 dollars per gallon indicate increasing global demands.

Because this dataset reports an area-level average for New York City, it smooths over station-to-station variation, differences across neighborhoods, and proximity to highways, giving a single representative price signal for the region. A drawback is that the EIA's NYC pricing area may not perfectly align with the full MTA service geography, which extends into parts of surrounding counties. A limitation that this dataset has compared to the primary data set is that it's based on a weekly basis instead of daily. So, the plan for primary and this secondary data set is to align them on a weekly basis. Since this data set has weekly reports, we will do this by aggregating daily MTA ridership into weekly totals.

(https://www.ebsco.com/research-starters/economics/2000s-oil-crisis)

(https://www.energy.gov/articles/what-eia-and-what-does-it-do#:~:text=The%20Energy%20Information%20Administration%20(EIA,the%20economy%20and%

In [13]:
```
# import pandas as pd

# Reads fuel price data
gas_raw = pd.read_csv('data/00-raw/Weekly_Gas_Prices.csv')
# drop extra columns generated by conversion XLS -> CSV
gas_raw = gas_raw.drop(gas_raw.columns[2:5], axis=1)
# Created data column
gas_raw['Date'] = pd.to_datetime(gas_raw['Date'])
# Filtered to just 2023
gas_2023 = gas_raw[(gas_raw['Date'] >= '2023-01-01') & (gas_raw['Date'] <= '2023-12
# Renamed column
gas_2023 = gas_2023.rename(columns={gas_2023.columns[1]: 'gas_price_dollars_per_gal
# Create csv of weekly prices
# gas_2023.to_csv('data/02-processed/nyc_gas_prices_weekly_2023.csv', index=False)
```

```
# Fill daily for each week
gas_daily_2023 = gas_2023.set_index('Date').resample('D').ffill().reset_index()
# Create csv of weekly prices
# gas_daily_2023.to_csv('data/02-processed/nyc_gas_prices_daily_2023.csv', index=Fa
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

# Results

## Exploratory Data Analysis

Instructions: REPLACE the contents of this cell with your work, including any updates to recover points lost in your EDA feedback

**NOTE** The final report should not be overly long. If this EDA step is huge and complicated and you went down a lot of dead ends, then you should do something about that. Some options...

1. remove any EDA you later decided was unecessary side quest.
2. if the code to EDA is very long (in either lines or in time to compute) make sure you write out images of the EDA into `results/` and reload them as images here instead of recalculating it. And make a statement here like "To see the code responsible for EDA analyses see `02-EDACheckpoint.ipynb` ."

## First Analysis You Did - Give it a better title

Some more words and stuff. Remember notebooks work best if you interleave the code that generates a result with properly annotate figures and text that puts these results into context. Every project will be different here... but some examples of analyses are: inferential statistics where you calculate p-values (like a t-test or a Monte Carlo simulation), regression models, or a machine learning model

**NOTE** The final report should not be overly long. If this analysis step is huge and complicated and you went down a lot of dead ends, then you should do something about that. Some options...

1. remove any analysis you later decided was unecessary side quest before checking in for the final hand in
2. if the code to do an analysis is very long (in either lines or in time to compute) put it in a different file. In that file make write out results of analysis into `results/` and reload

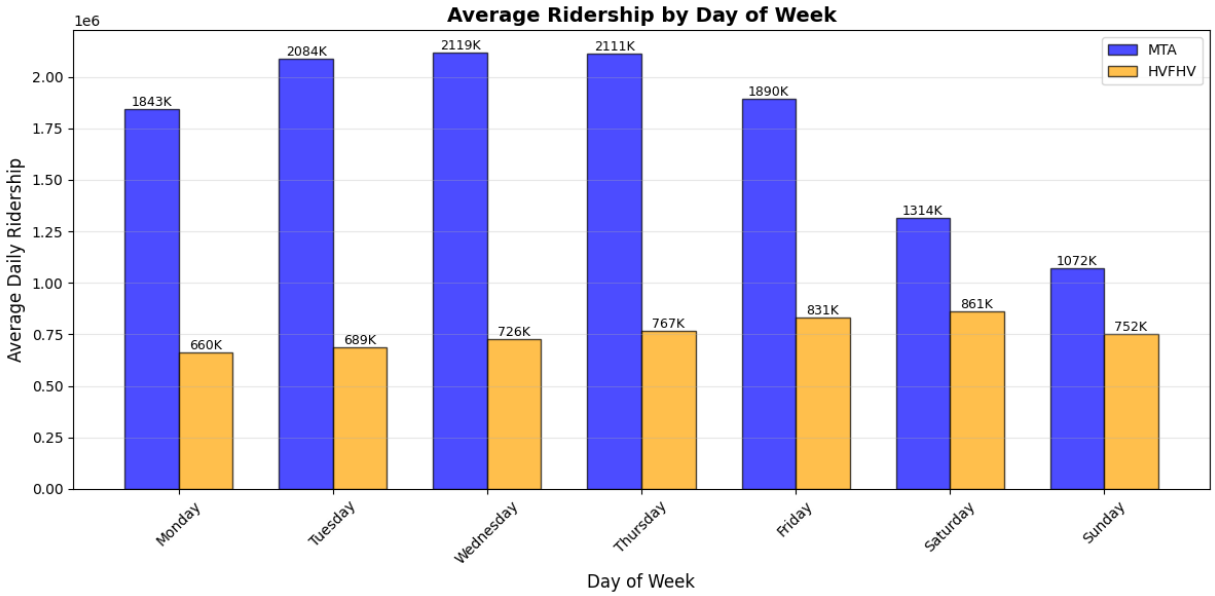
the results here instead of recalculating it. And make a statement here like "To see the code responsible for analyses see `path/to/some-file-name.ipynb_or.py` ."

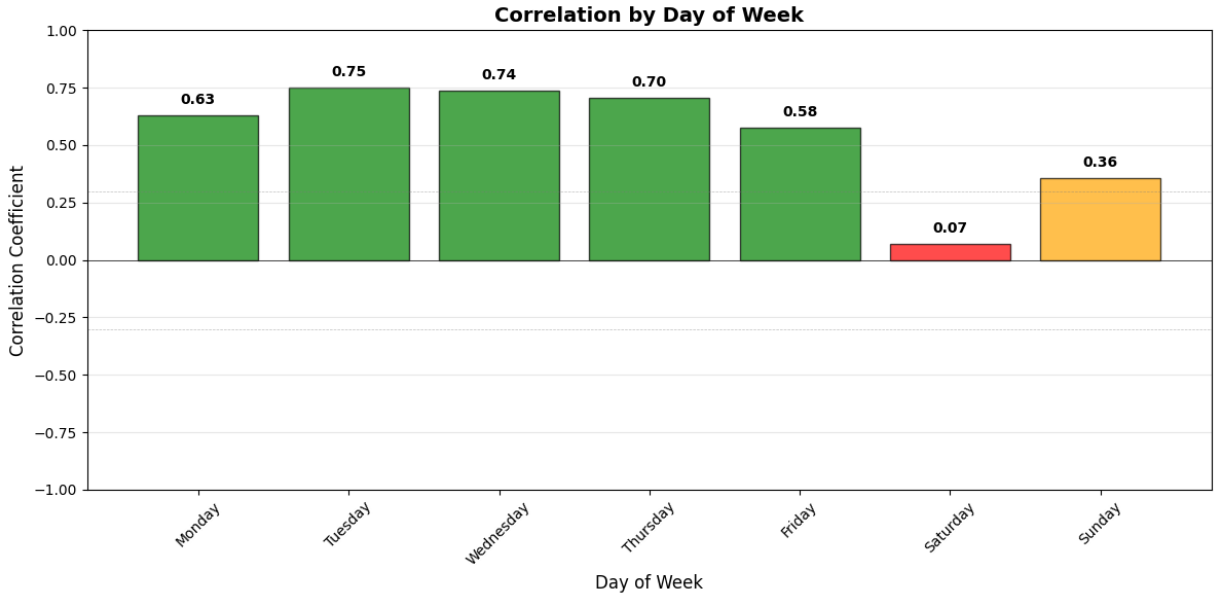
## Section 1 of EDA - Ridehailing vs MTA ridership analysis

This section should include the analysis of the subway and ridehailing datasets. The map of ny with stations and zones should go here, etc.

In [14]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import geopandas as gpd
from shapely.geometry import Point, box
from shapely.ops import unary_union
from datetime import datetime
from modules import graph, utils, maps

subway_stations_gdf, taxi_zones_gdf, subway_lines_gdf = utils.prepare_gdf_data()
ridehail_df, mta_df = utils.prepare_ridership_data('data/02-processed/ridehailing_d
```

In [32]:
```
from modules import graph
ridehail_df, mta_df = utils.prepare_ridership_data('data/02-processed/ridehailing_d
#print(ridehail_df.columns)

graph.plot_ridership_histogram(mta_df, ridehail_df)
```





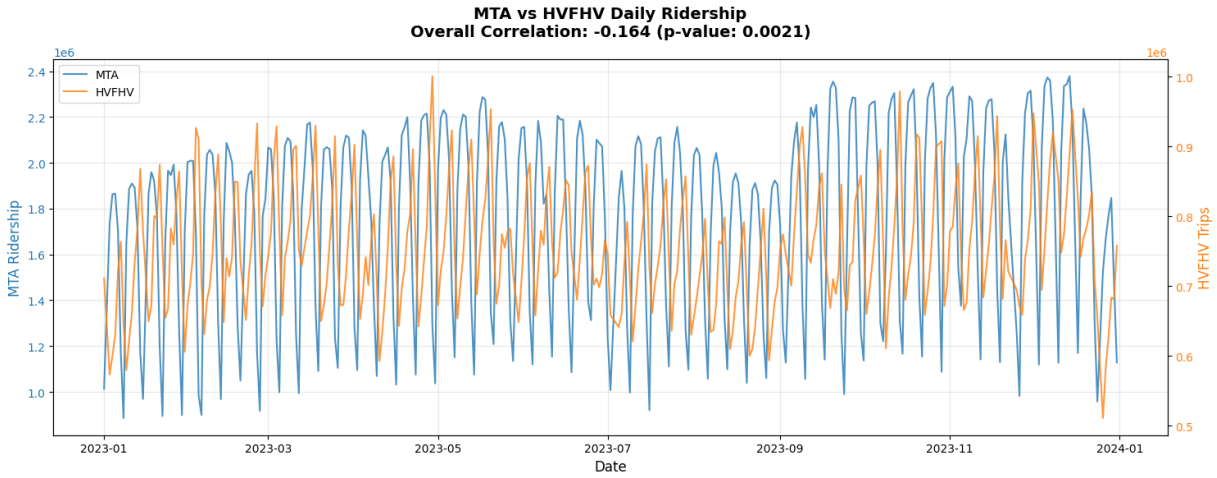

From this histogram we can observe that the MTA sees a sharp decrease in ridership over the weekend on average, whereas ridehailing services increase over the weekend indicating that there might be a negative correlation.

From the correlation by day graph, we observe that there are moderate-strong levels of correlation throughout each weekday, but the correlation drops significantly on the weekend

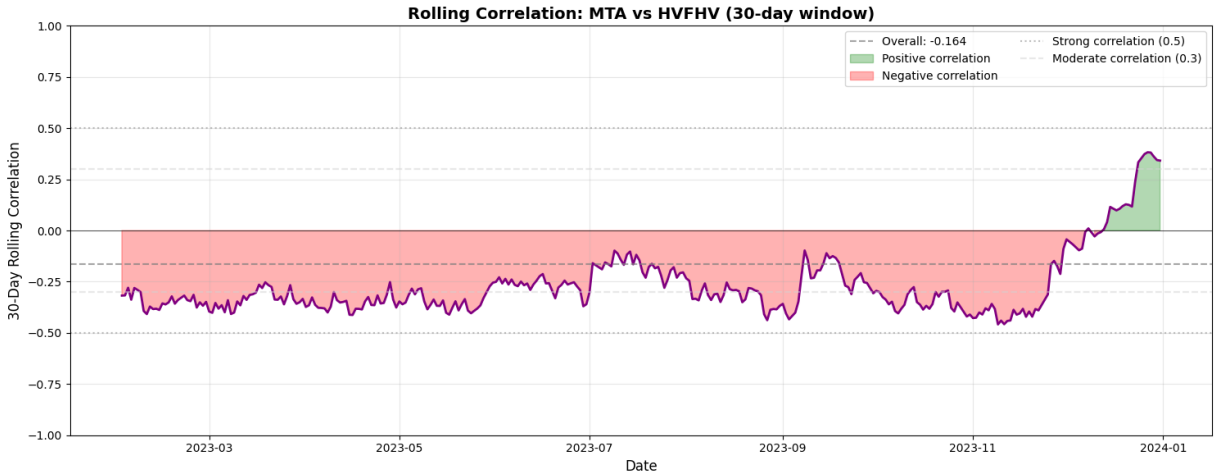
In [ ]:

In [36]:
```
graph.plot_mta_vs_ridehail_both(mta_df, ridehail_df)
scatterplot = graph.scatterplot_mta_vs_ridehail_daily(mta_df, ridehail_df)
```

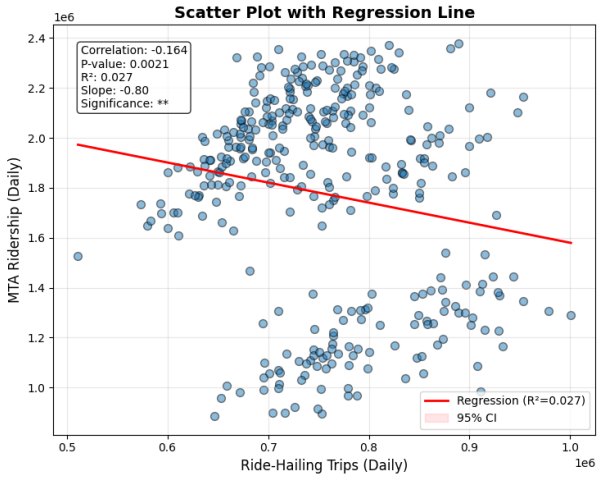
```
======================================================================
PLOT 1: Daily Time Series Comparison
======================================================================
```

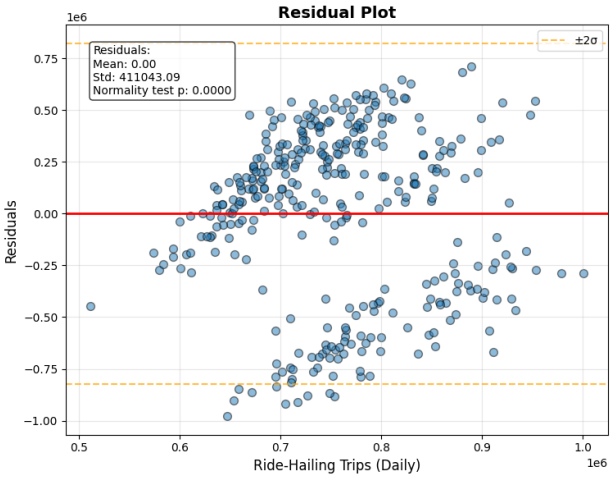


```
📊 Daily Time Series Analysis:
==================================================
Overall Pearson correlation: -0.1643
P-value: 0.0021
Significance: ** (p < 0.01)

======================================================================
PLOT 2: Rolling Correlation (30-day window)
======================================================================
```

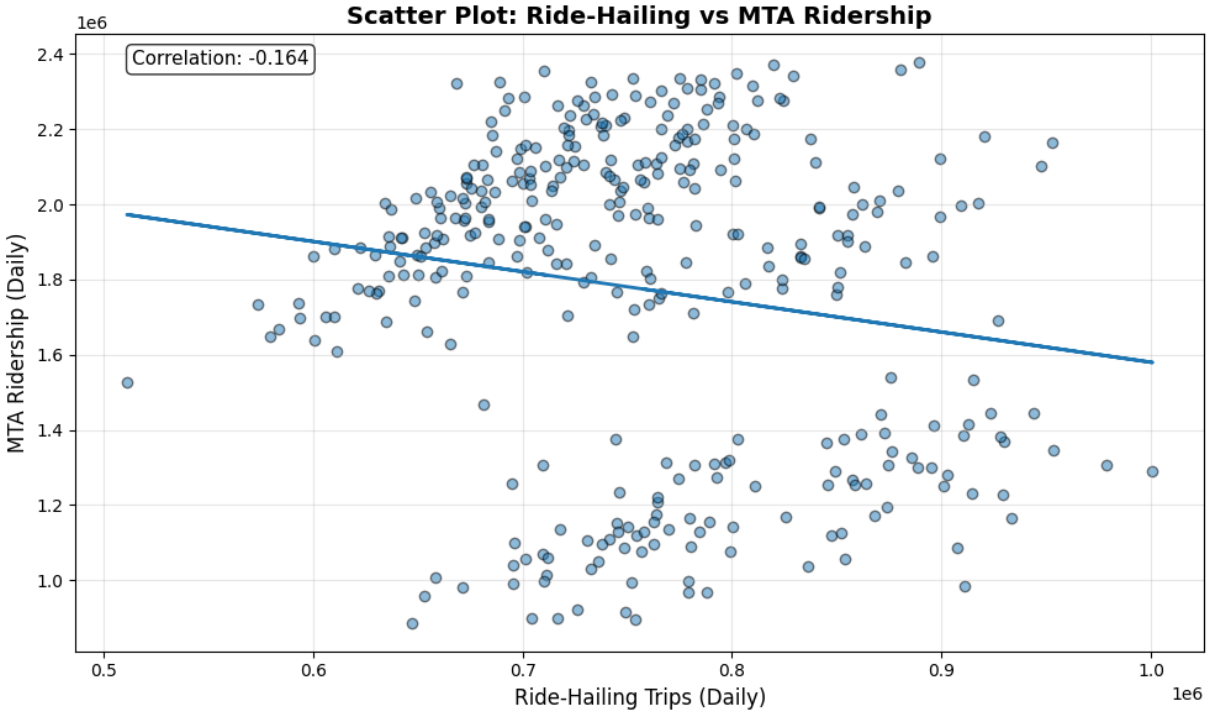



```
📊 Rolling Correlation Analysis (30-day window):
==================================================
Rolling correlation statistics:
  - Mean: -0.2743
  - Std: 0.1496
  - Min: -0.4587
  - Max: 0.3822
  - % time positive: 5.2%
  - % time strong (>0.5): 0.0%
```



```
📊 Detailed Correlation Analysis:
==================================================
Pearson correlation coefficient: -0.1643
P-value: 0.0021
R-squared: 0.0270

Regression equation: MTA = -0.80 × HVFHV + 2383789.26
Standard error: 0.2591

Significance level:
  ** p < 0.01 (Very significant)

Residual analysis:
  Mean: 0.0000 (should be close to 0)
  Standard deviation: 411043.0934
  Normality test p-value: 0.0000
```




```
📊 Correlation between ride-hailing and MTA ridership: -0.164
```

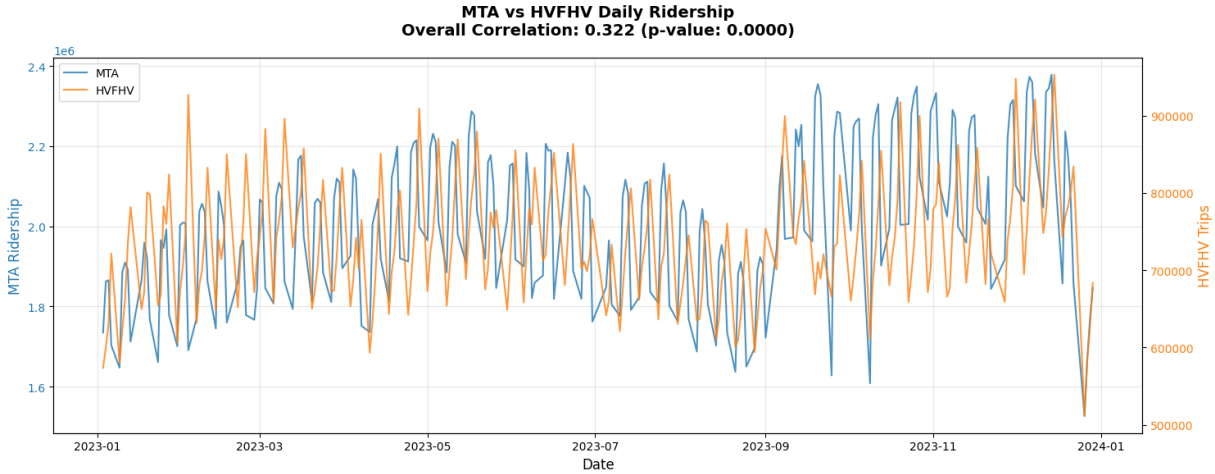
On this graph we track the MTA vs HVFHV ridership over time, and see that they both fluctuate weekly, we also observed that the overall correlation is -0.164, which means there's probably no correlation. But after looking at the scatter plot, we can observe that there are 2 clusters of data, I hypothesize that it's weekday and weekend.

Let's seperate weekday and weekend and see if we can find any correlation

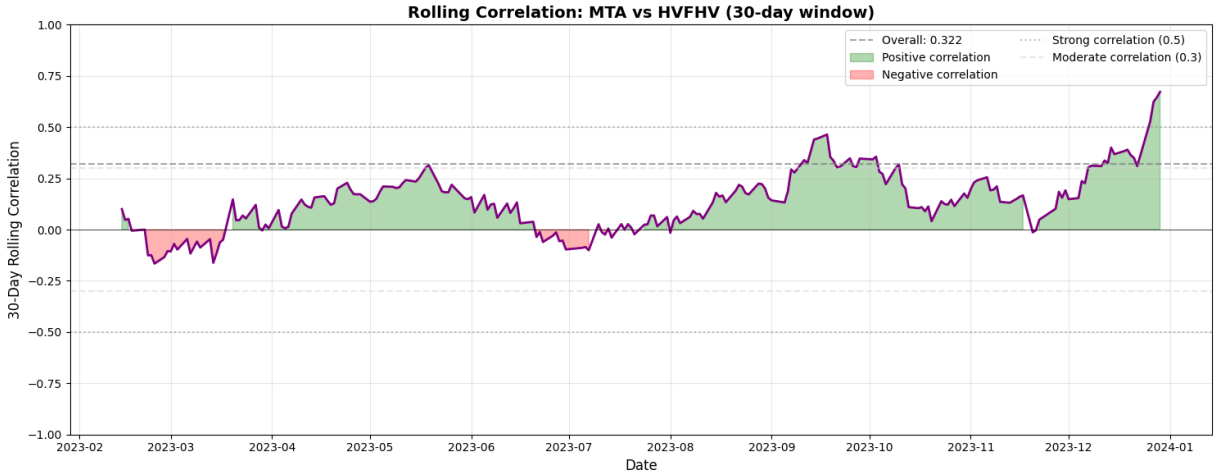
In [ ]:
```
mta_df_weekday = mta_df[mta_df['weekend'] ==False]
ridehail_df_weekday = ridehail_df[ridehail_df['weekend'] == False]

mta_df_weekend = mta_df[mta_df['weekend'] ==True]
ridehail_df_weekend =ridehail_df[ridehail_df['weekend'] == True]

print('Weekday MTA vs Ridehail')
graph.plot_mta_vs_ridehail_both(mta_df_weekday, ridehail_df_weekday)
```

```
graph.scatterplot_mta_vs_ridehail_daily(mta_df_weekday, ridehail_df_weekday)
print('Weekend MTA vs Ridehail')
graph.plot_mta_vs_ridehail_both(mta_df_weekend, ridehail_df_weekend)
graph.scatterplot_mta_vs_ridehail_daily(mta_df_weekend, ridehail_df_weekend)
```

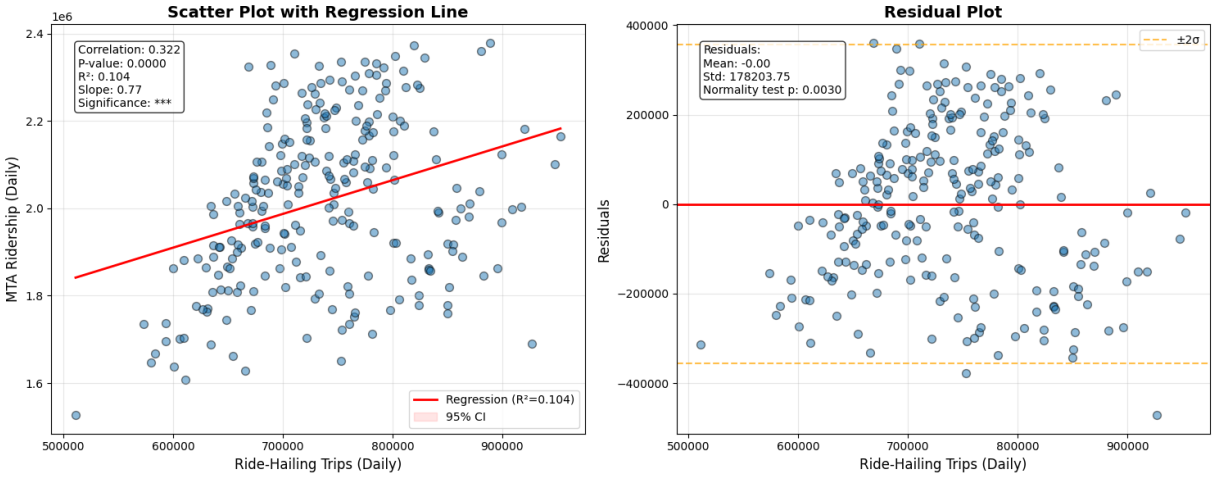
```
Weekday MTA vs Ridehail
======================================================================
PLOT 1: Daily Time Series Comparison
======================================================================
```



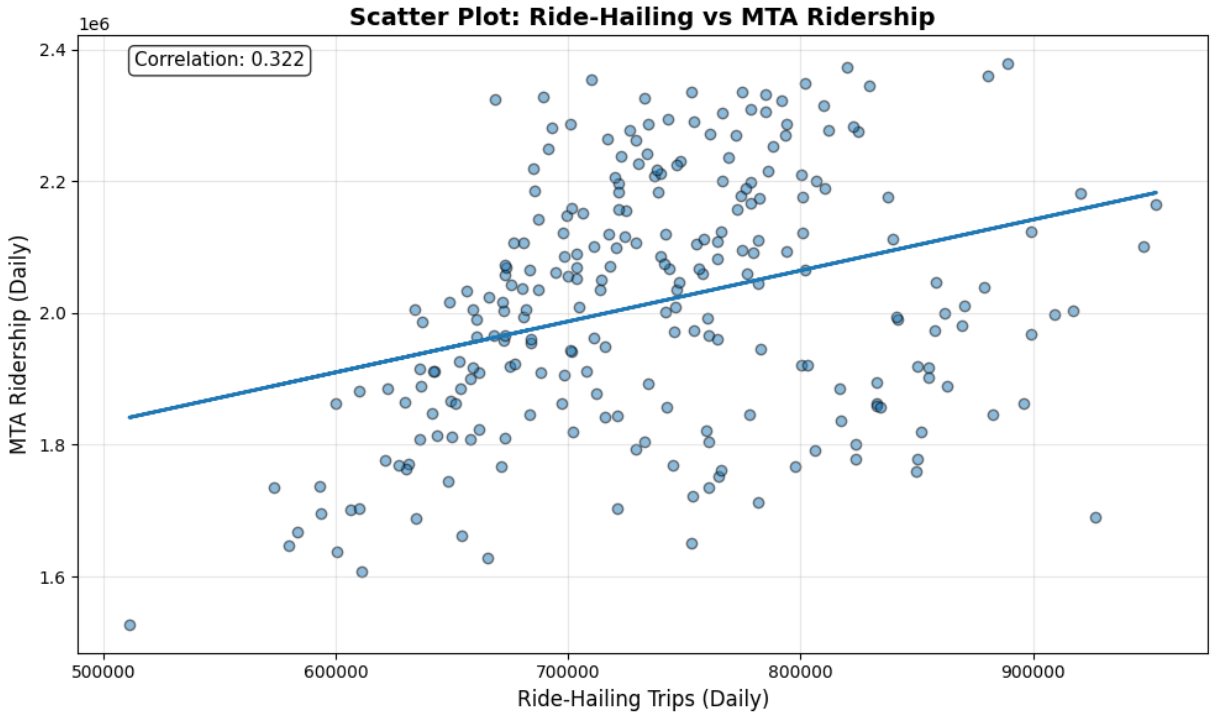

```
📊 Daily Time Series Analysis:
==================================================
Overall Pearson correlation: 0.3221
P-value: 0.0000
Significance: *** (p < 0.001)

======================================================================
PLOT 2: Rolling Correlation (30-day window)
======================================================================
```



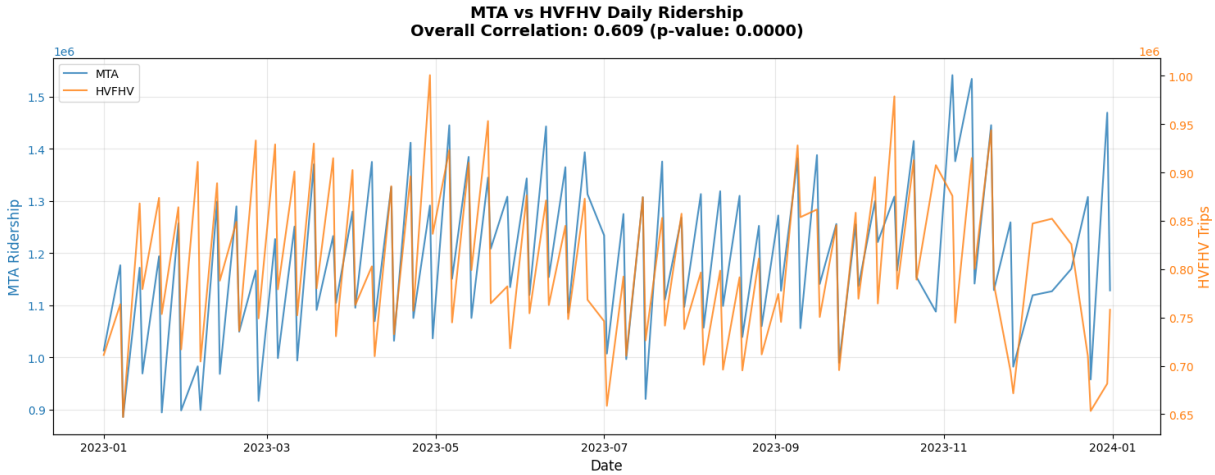

```
📊 Rolling Correlation Analysis (30-day window):
==================================================
Rolling correlation statistics:
  - Mean: 0.1397
  - Std: 0.1486
  - Min: -0.1659
  - Max: 0.6720
  - % time positive: 72.2%
  - % time strong (>0.5): 1.6%
```



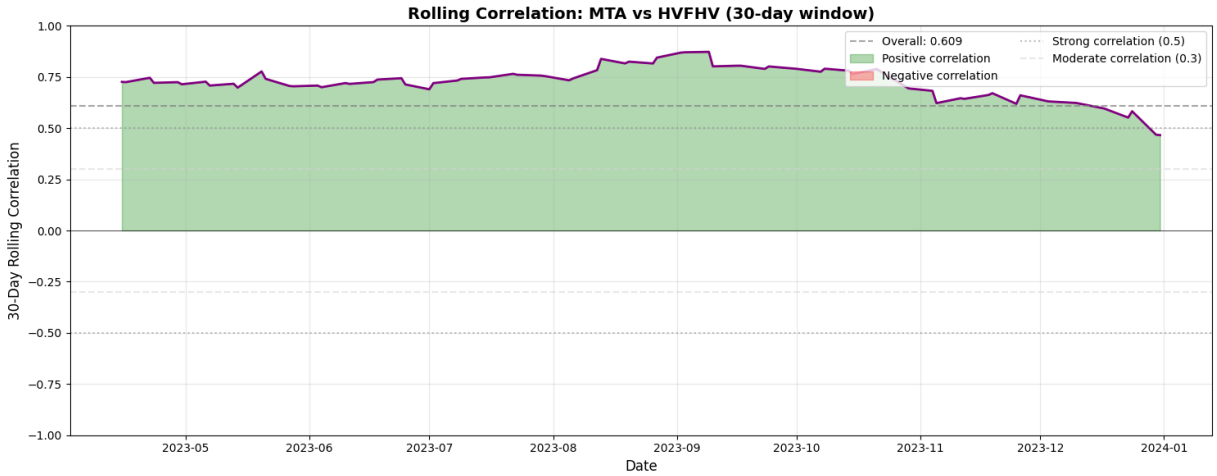

```
📊 Detailed Correlation Analysis:
==================================================
Pearson correlation coefficient: 0.3221
P-value: 0.0000
R-squared: 0.1037

Regression equation: MTA = 0.77 × HVFHV + 1446520.82
Standard error: 0.1447

Significance level:
  *** p < 0.001 (Highly significant)

Residual analysis:
  Mean: -0.0000 (should be close to 0)
  Standard deviation: 178203.7519
  Normality test p-value: 0.0030
```



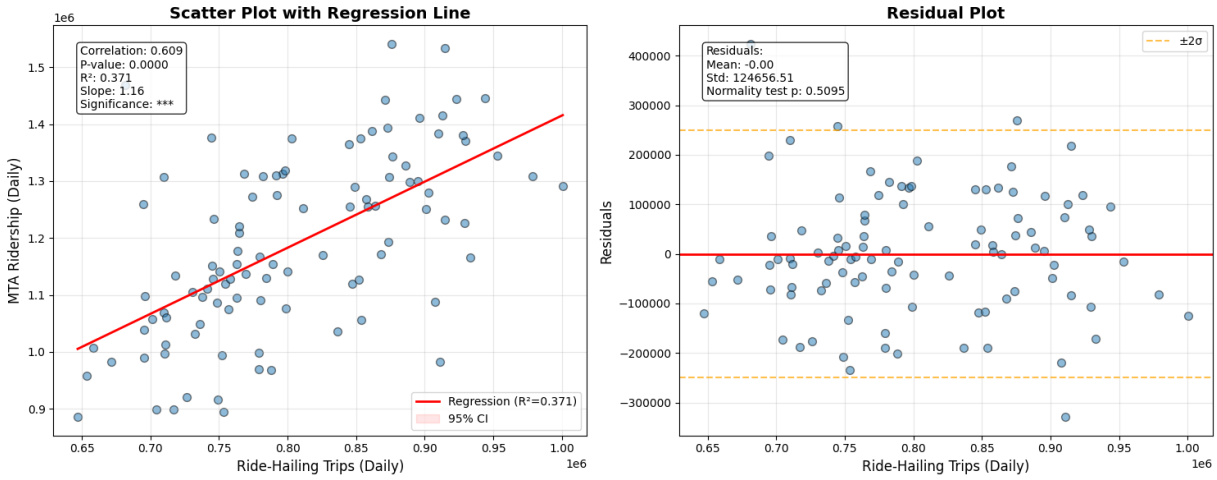
```
📊 Correlation between ride-hailing and MTA ridership: 0.322
Weekend MTA vs Ridehail
======================================================================
PLOT 1: Daily Time Series Comparison
======================================================================
```



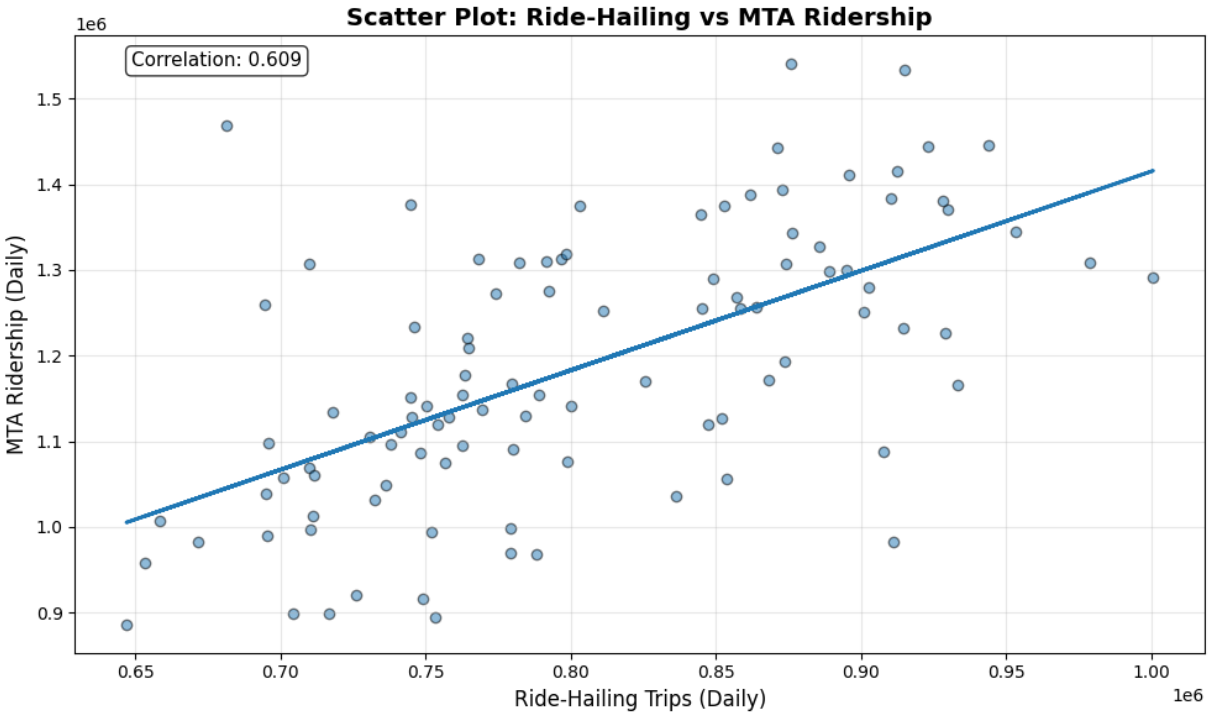

```
📊 Daily Time Series Analysis:
==================================================
Overall Pearson correlation: 0.6087
P-value: 0.0000
Significance: *** (p < 0.001)

======================================================================
PLOT 2: Rolling Correlation (30-day window)
======================================================================
```




```
📊 Rolling Correlation Analysis (30-day window):
==================================================
Rolling correlation statistics:
  - Mean: 0.7279
  - Std: 0.0804
  - Min: 0.4661
  - Max: 0.8728
  - % time positive: 71.3%
  - % time strong (>0.5): 69.3%
```




```
📊 Detailed Correlation Analysis:
==================================================
Pearson correlation coefficient: 0.6087
P-value: 0.0000
R-squared: 0.3705

Regression equation: MTA = 1.16 × HVFHV + 253527.25
Standard error: 0.1522

Significance level:
  *** p < 0.001 (Highly significant)

Residual analysis:
  Mean: -0.0000 (should be close to 0)
  Standard deviation: 124656.5143
  Normality test p-value: 0.5095
```




```
📊 Correlation between ride-hailing and MTA ridership: 0.609
```

# Zonal Analysis

For the purposes of readability, I have stored the code for these functions in modules/map.py as they got quite large. Below are the heatmaps for some of the days of the week.

In [ ]:
```
from modules import maps


ridehail_df['PULocationID'] = ridehail_df['PULocationID'].astype(str)
taxi_zones_gdf['locationid'] = taxi_zones_gdf['locationid'].astype(str)

weekday_df = ridehail_df[ridehail_df['weekend'] == 0]
weekend_df = ridehail_df[ridehail_df['weekend'] == 1]

weekday_avg = weekday_df.groupby('PULocationID')['trip_count'].mean().reset_index()
weekend_avg = weekend_df.groupby('PULocationID')['trip_count'].mean().reset_index()
weekday_avg.columns = ['PULocationID', 'avg_trip_count']
weekend_avg.columns = ['PULocationID', 'avg_trip_count']

zones_weekday = taxi_zones_gdf.merge(weekday_avg, left_on='locationid', right_on='P
zones_weekday['avg_trip_count'] = zones_weekday['avg_trip_count'].fillna(0)
manhattan = zones_weekday[zones_weekday['borough'] == 'Manhattan']

print(f"\n Weekday Ridehail- Manhattan Summary:")
print(f"Total avg trips: {manhattan['avg_trip_count'].sum():,.0f}")
print("\nTop 5 zones:")
top = manhattan.nlargest(5, 'avg_trip_count')[['zone', 'avg_trip_count']]
for idx, row in top.iterrows():
    print(f"  {row['zone']}: {row['avg_trip_count']:,.0f} avg trips")


zones_weekend = taxi_zones_gdf.merge(weekend_avg, left_on='locationid', right_on='P
zones_weekend['avg_trip_count'] = zones_weekend['avg_trip_count'].fillna(0)
manhattan = zones_weekend[zones_weekend['borough'] == 'Manhattan']

print(f"\n Weekend Ridehail- Manhattan Summary:")
print(f"Total avg trips: {manhattan['avg_trip_count'].sum():,.0f}")
print("\nTop 5 zones:")
top = manhattan.nlargest(5, 'avg_trip_count')[['zone', 'avg_trip_count']]
for idx, row in top.iterrows():
    print(f"  {row['zone']}: {row['avg_trip_count']:,.0f} avg trips")

#maps.plot_ridehail_heatmap_by_day(ridehail_df, taxi_zones_gdf, 1, 20000, False)
#maps.plot_ridehail_heatmap_by_day(ridehail_df, taxi_zones_gdf, 2, 20000, False)
#maps.plot_ridehail_heatmap_by_day(ridehail_df, taxi_zones_gdf, 3, 20000, False)
#maps.plot_ridehail_heatmap_by_day(ridehail_df, taxi_zones_gdf, 4, 20000, False)
#maps.plot_ridehail_heatmap_by_day(ridehail_df, taxi_zones_gdf, 5, 20000, False)
#maps.plot_ridehail_heatmap_by_day(ridehail_df, taxi_zones_gdf, 6, 20000, False)
#maps.plot_ridehail_heatmap_by_day(ridehail_df, taxi_zones_gdf, 7, 20000, False)
```

```
 weekday - Manhattan Summary:
Total avg trips: 347,704

Top 5 zones:
  Midtown Center: 14,503 avg trips
  Times Sq/Theatre District: 11,779 avg trips
  Upper East Side South: 11,150 avg trips
  Midtown East: 10,025 avg trips
  Upper East Side North: 9,964 avg trips

 weekend - Manhattan Summary:
Total avg trips: 372,083

Top 5 zones:
  East Village: 17,234 avg trips
  Lower East Side: 12,607 avg trips
  West Village: 11,649 avg trips
  Clinton East: 11,273 avg trips
  Times Sq/Theatre District: 11,106 avg trips
```

Above we've identified the Top Zones for Ridehailing Journeys

| Weekday | Weekend |
|---|---|
| Midtown Center: 14,503 avg trips | East Village: 17,234 avg trips |
| Times Sq/Theatre District: 11,779 avg trips | Lower East Side: 12,607 avg trips |
| Upper East Side South: 11,150 avg trips | West Village: 11,649 avg trips |
| Midtown East: 10,025 avg trips | Clinton East: 11,273 avg trips |
| Upper East Side North: 9,964 avg trips | Times Sq/Theatre District: 11,106 avg trips |

In [30]:
```python
from modules import maps, utils

# Load the data
subway_stations_gdf, taxi_zones_gdf, subway_lines_gdf = utils.prepare_gdf_data()
ridehail_df, mta_df = utils.prepare_ridership_data(
    'data/02-processed/ridehailing_daily_cleaned.csv',
    'data/02-processed/MTA_Ridership_cleaned.csv'
)

# ============================================
# MAP SUBWAY STATIONS TO TAXI ZONES (same as heatmap function)
# ============================================

# Make a copy and ensure CRS
stations_gdf = subway_stations_gdf.copy()
if stations_gdf.crs is None:
    stations_gdf = stations_gdf.set_crs('EPSG:4326')

# Convert to projected CRS for buffering
stations_gdf = stations_gdf.to_crs('EPSG:3857')
taxi_zones_proj = taxi_zones_gdf.to_crs('EPSG:3857')
```

```
# Create buffers (0.1 miles - same as heatmap)
buffer_miles = 0.1
buffer_meters = buffer_miles * 1609.34
stations_gdf['buffer_geom'] = stations_gdf.geometry.buffer(buffer_meters)

# Spatial join using buffers
stations_buffer = stations_gdf[['Complex ID', 'Stop Name', 'Borough']].copy()
stations_buffer = stations_buffer.set_geometry(stations_gdf['buffer_geom'])
stations_buffer.crs = stations_gdf.crs

station_zone_mapping = gpd.sjoin(
    stations_buffer,
    taxi_zones_proj,
    how='left',
    predicate='intersects'
)

# Remove duplicates
station_zone_mapping = station_zone_mapping.drop_duplicates(subset=['Complex ID', '

# Prepare MTA data
mta_df['date'] = pd.to_datetime(mta_df['date'])
mta_df['station_complex_id'] = mta_df['station_complex_id'].astype(str)
station_zone_mapping['Complex ID'] = station_zone_mapping['Complex ID'].astype(str)
station_zone_mapping['locationid'] = station_zone_mapping['locationid'].astype(str)
taxi_zones_gdf['locationid'] = taxi_zones_gdf['locationid'].astype(str)

# ============================================
# MTA ANALYSIS BY TAXI ZONE
# ============================================

# Merge ridership with zone mapping for all data
mta_with_zones = station_zone_mapping.merge(
    mta_df,
    left_on='Complex ID',
    right_on='station_complex_id',
    how='left'
)

# Calculate average ridership by taxi zone for weekdays/weekends
weekday_mta_zones = mta_with_zones[mta_with_zones['weekend'] == 0].groupby('locatio
    'ridership': 'mean'
}).reset_index()

weekend_mta_zones = mta_with_zones[mta_with_zones['weekend'] == 1].groupby('locatio
    'ridership': 'mean'
}).reset_index()

weekday_mta_zones.columns = ['locationid', 'avg_ridership']
weekend_mta_zones.columns = ['locationid', 'avg_ridership']

# Merge with taxi zones
zones_weekday = taxi_zones_gdf.merge(weekday_mta_zones, on='locationid', how='left'
zones_weekday['avg_ridership'] = zones_weekday['avg_ridership'].fillna(0)
manhattan_weekday = zones_weekday[zones_weekday['borough'] == 'Manhattan']
```

```
print(f"\n weekday - MTA Manhattan Summary (by taxi zone):")
print(f"Total avg ridership: {manhattan_weekday['avg_ridership'].sum():,.0f}")
print("\nTop 5 zones:")
top = manhattan_weekday.nlargest(5, 'avg_ridership')[['zone', 'avg_ridership']]
for idx, row in top.iterrows():
    print(f"  {row['zone']}: {row['avg_ridership']:,.0f} avg riders")

zones_weekend = taxi_zones_gdf.merge(weekend_mta_zones, on='locationid', how='left'
zones_weekend['avg_ridership'] = zones_weekend['avg_ridership'].fillna(0)
manhattan_weekend = zones_weekend[zones_weekend['borough'] == 'Manhattan']

print(f"\n weekend - MTA Manhattan Summary (by taxi zone):")
print(f"Total avg ridership: {manhattan_weekend['avg_ridership'].sum():,.0f}")
print("\nTop 5 zones:")
top = manhattan_weekend.nlargest(5, 'avg_ridership')[['zone', 'avg_ridership']]
for idx, row in top.iterrows():
    print(f"  {row['zone']}: {row['avg_ridership']:,.0f} avg riders")

# Calculate percent change
weekday_total = manhattan_weekday['avg_ridership'].sum()
weekend_total = manhattan_weekend['avg_ridership'].sum()
pct_change = ((weekend_total - weekday_total) / weekday_total) * 100
print(f"\n📈 MTA Weekend vs Weekday change (by zone): {pct_change:+.1f}%")

# Uncomment if you want to generate heatmaps
# maps.plot_subway_heatmap_by_day(mta_df, subway_stations_gdf, taxi_zones_gdf, 1, 7
# maps.plot_subway_heatmap_by_day(mta_df, subway_stations_gdf, taxi_zones_gdf, 2, 7
# maps.plot_subway_heatmap_by_day(mta_df, subway_stations_gdf, taxi_zones_gdf, 3, 7
# maps.plot_subway_heatmap_by_day(mta_df, subway_stations_gdf, taxi_zones_gdf, 4, 7
# maps.plot_subway_heatmap_by_day(mta_df, subway_stations_gdf, taxi_zones_gdf, 5, 7
# maps.plot_subway_heatmap_by_day(mta_df, subway_stations_gdf, taxi_zones_gdf, 6, 7
# maps.plot_subway_heatmap_by_day(mta_df, subway_stations_gdf, taxi_zones_gdf, 7, 7
```

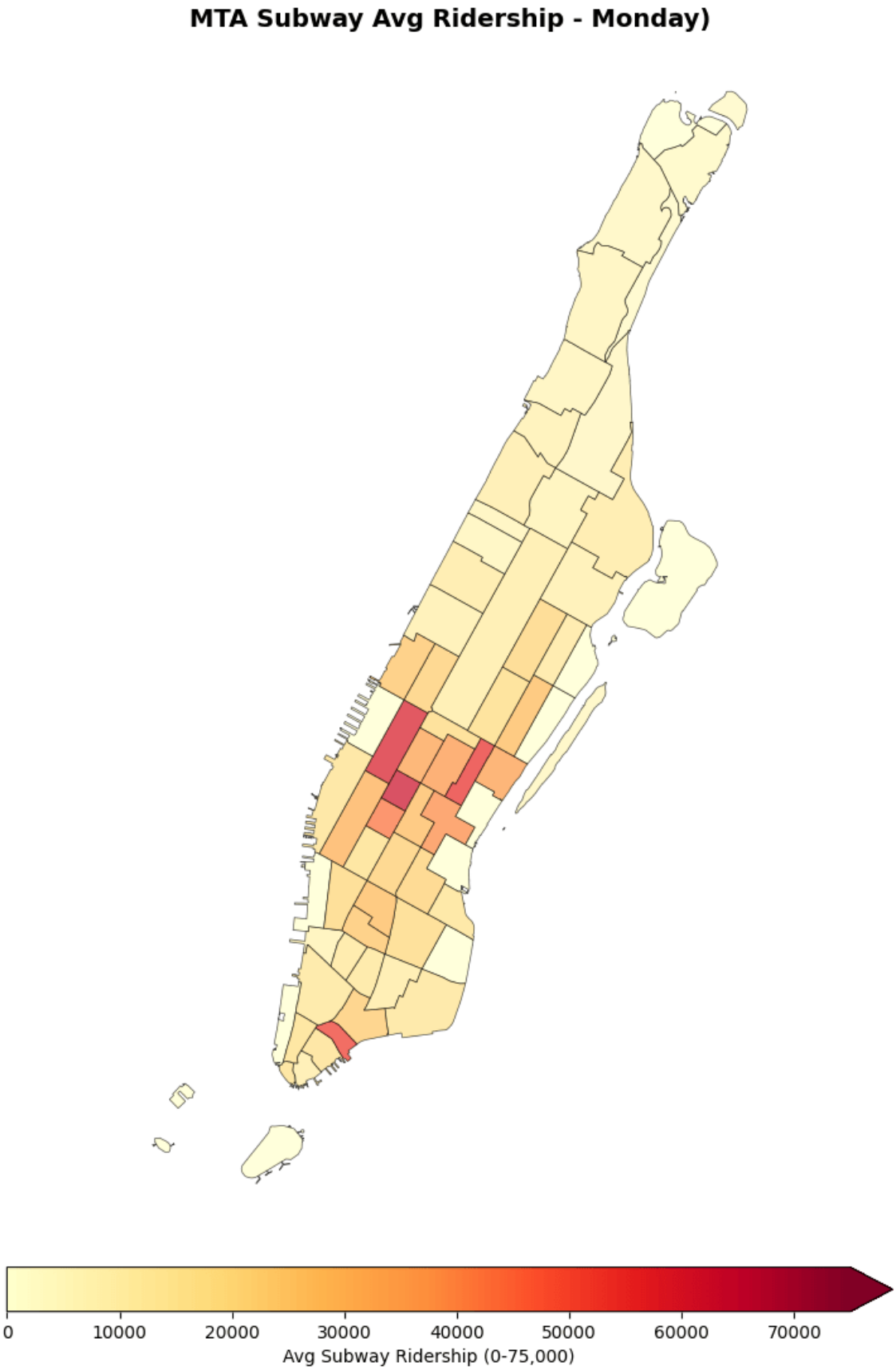
```
 weekday - MTA Manhattan Summary (by taxi zone):
Total avg ridership: 1,200,115

Top 5 zones:
  Garment District: 68,604 avg riders
  Clinton East: 65,275 avg riders
  Midtown East: 59,156 avg riders
  Seaport: 58,126 avg riders
  Penn Station/Madison Sq West: 46,142 avg riders

 weekend - MTA Manhattan Summary (by taxi zone):
Total avg ridership: 709,363

Top 5 zones:
  Clinton East: 43,399 avg riders
  Garment District: 42,288 avg riders
  Seaport: 29,213 avg riders
  Penn Station/Madison Sq West: 28,200 avg riders
  Midtown East: 27,126 avg riders

📈 MTA Weekend vs Weekday change (by zone): -40.9%
```

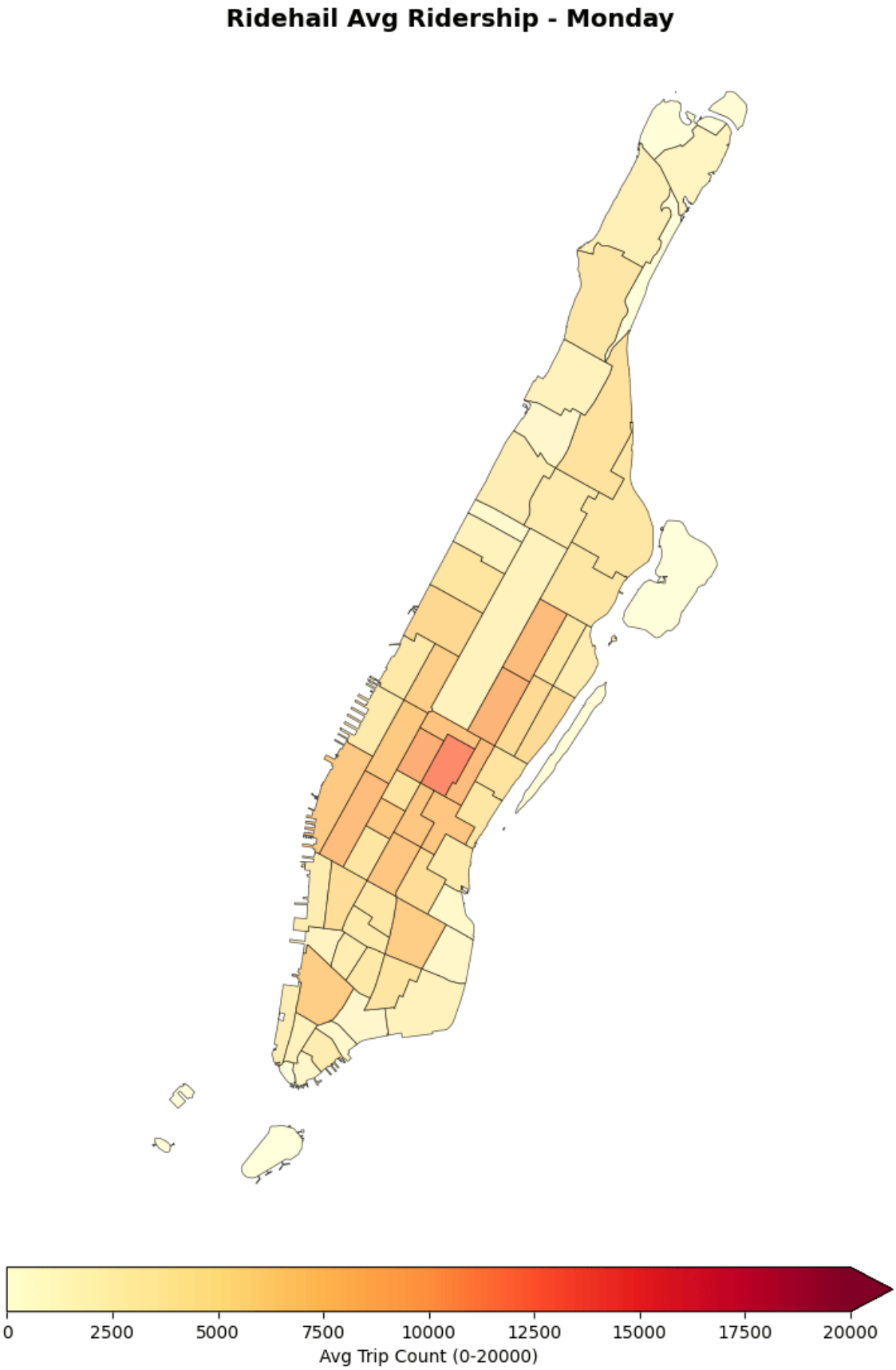
Above we've identified the top zones for the weekday and weekend

| Weekday | Weekend |
|---|---|
| Garment District: 68,604 avg riders | Clinton East: 43,399 avg riders |
| Clinton East: 65,275 avg riders | Garment District: 42,288 avg riders |
| Midtown East: 59,156 avg riders | Seaport: 29,213 avg riders |
| Seaport: 58,126 avg riders | Penn Station/Madison Sq West: 28,200 avg riders |
| Penn Station/Madison Sq West: 46,142 avg riders | Midtown East: 27,126 avg riders |

As we can see The most popular stations are the same.

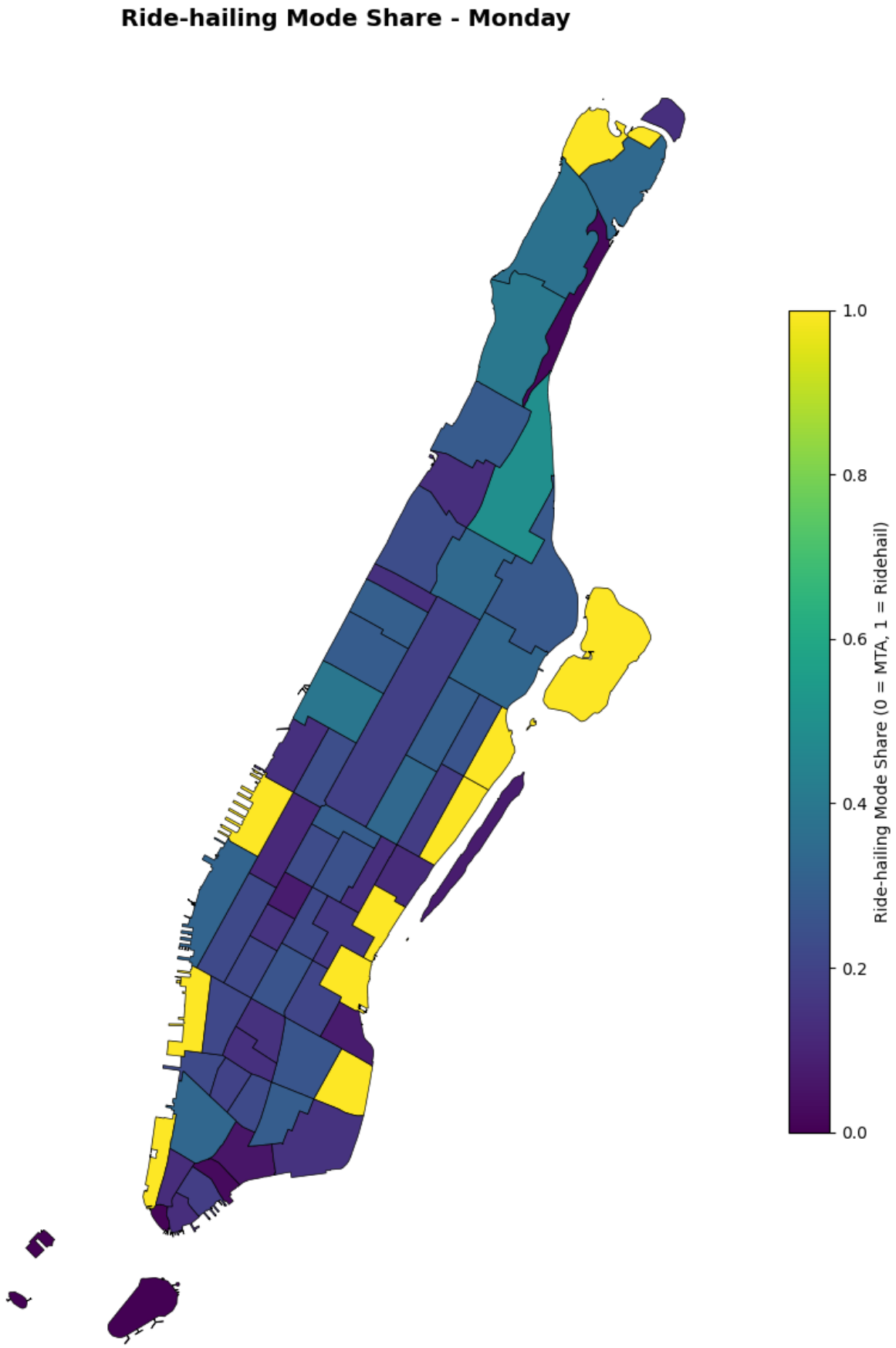
Below are the days of the week.

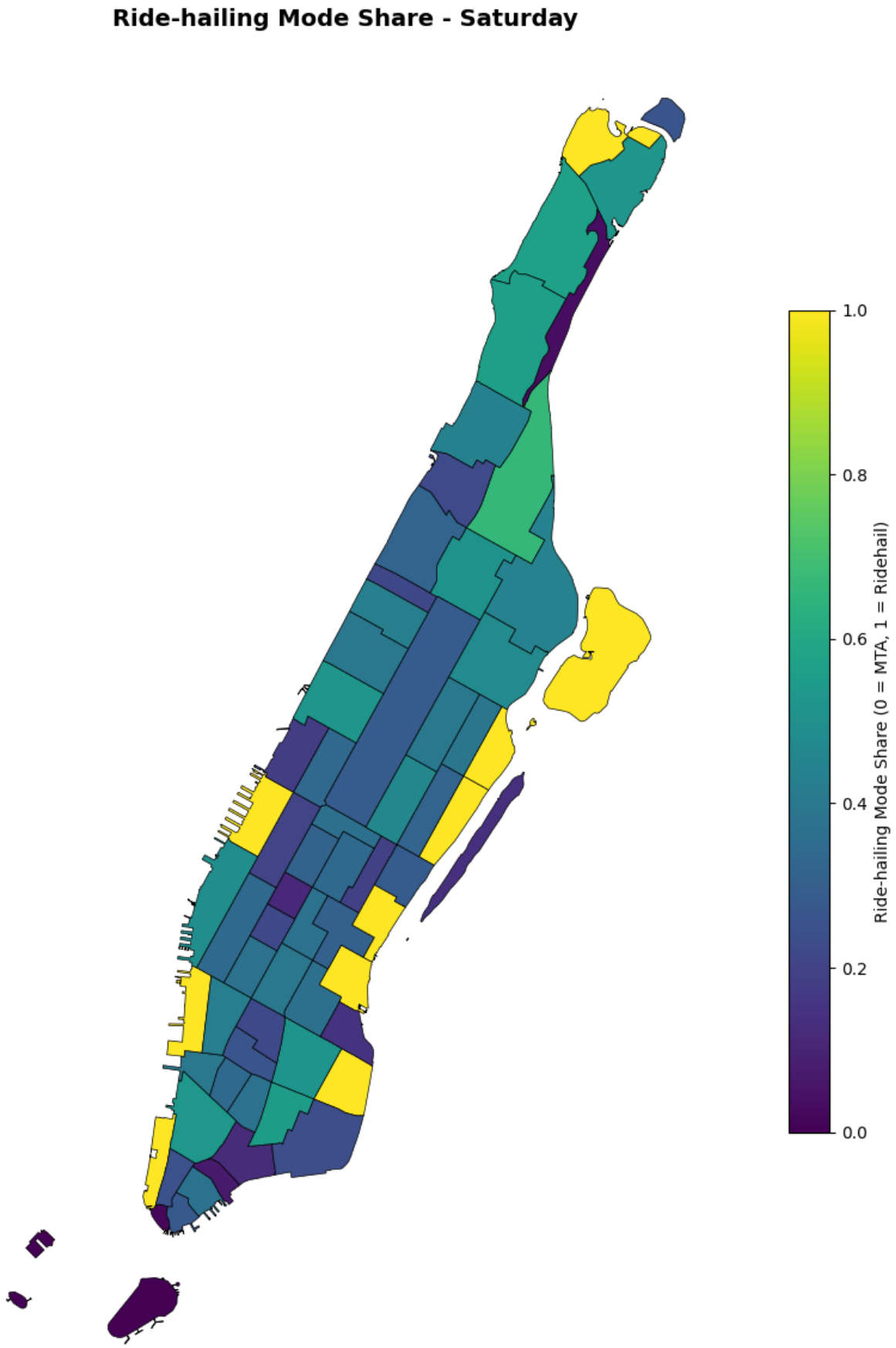

MTA Subway Avg Ridership - Monday)
0
10000
20000
30000
40000
50000
60000
70000
Avg Subway Ridership (0-75,000)




After mapping subway stations to complex zones, we created a heatmap for subway ridership, and found that ridership remains stagnant during the weekend.

In [29]:
```
modal = maps.plot_mode_share_heatmap_by_day(ridehail_df, mta_df, subway_stations_gd
modal = maps.plot_mode_share_heatmap_by_day(ridehail_df, mta_df, subway_stations_gd
```


Ride-hailing Mode Share - Monday
1.0
0.8
0.6
0.4
0.2
0.0
Ride-hailing Mode Share (0 = MTA, 1 = Ridehail)


Ride-hailing Mode Share - Saturday
1.0
0.8
0.6
0.4
0.2
0.0
Ride-hailing Mode Share (0 = MTA, 1 = Ridehail)

## Weather and Gas EDA

After reviewing the trend of weekly subway ridership and gas prices we found that there is virtually no correlation (r = 0.02) in the year 2023. The gas prices ranged from $3.125 to $3.767, parts of the year surpassing our hypothesized threshold. Thus, our initial hypothesis was incorrect as there is no correlation between gas prices and subway ridership.

Note, in this section, we seperate weekdays and weekends due to the large shift in riders between weekdays and weekends.

Rain & Snow Analysis: From our rain analysis we see that there is a slightly significant negative correlation (r = -0.32, r = -0.14) between rain and ridership when separating the weekdays and weekends as well as removing our list of outliers. For snow we note that no snow days occurred on weekends and the correlation on the weekdays is far less impactful than rain (r = -0.09).

We take the average daily temperature and daily riders to avoid the influence of the time of day, and all the activities and habits of people surrounding that.

We hypothesized a lower threshold of 50°F (10°C) and an upper threshold of 80°F (27°C), a lower or higher temperature than those thresholds resulting in a shift of riders from subways to ride hailing.

Overall, our original idea of ridership decreasing past 50°F and 80°F was correct, however, the data does not reflect the hypothesized thresholds. Instead, the data shows peaks; ridership and number of rides decreasing as temperature drifts from this peak.

Peaks in ridership were around 57-63.5°F (roughly 14-17°C) for both ride-hailing and subway modes indicating that travel demand is highest in mild, comfortable temperatures where outdoor activity is abundant.

Visually, the graph for ride hailing is much steeper than the subway's. This suggests that ride hailing users are more sensitive to temperature shifts, changes in temperature leading to greater changes in rides as compared to subway riders, while subways have more regular commuters. These commuters are most likely workers as the subway experiences more riders during weekdays and less over weekends; ride hailing holds a similar amount of riders over both.

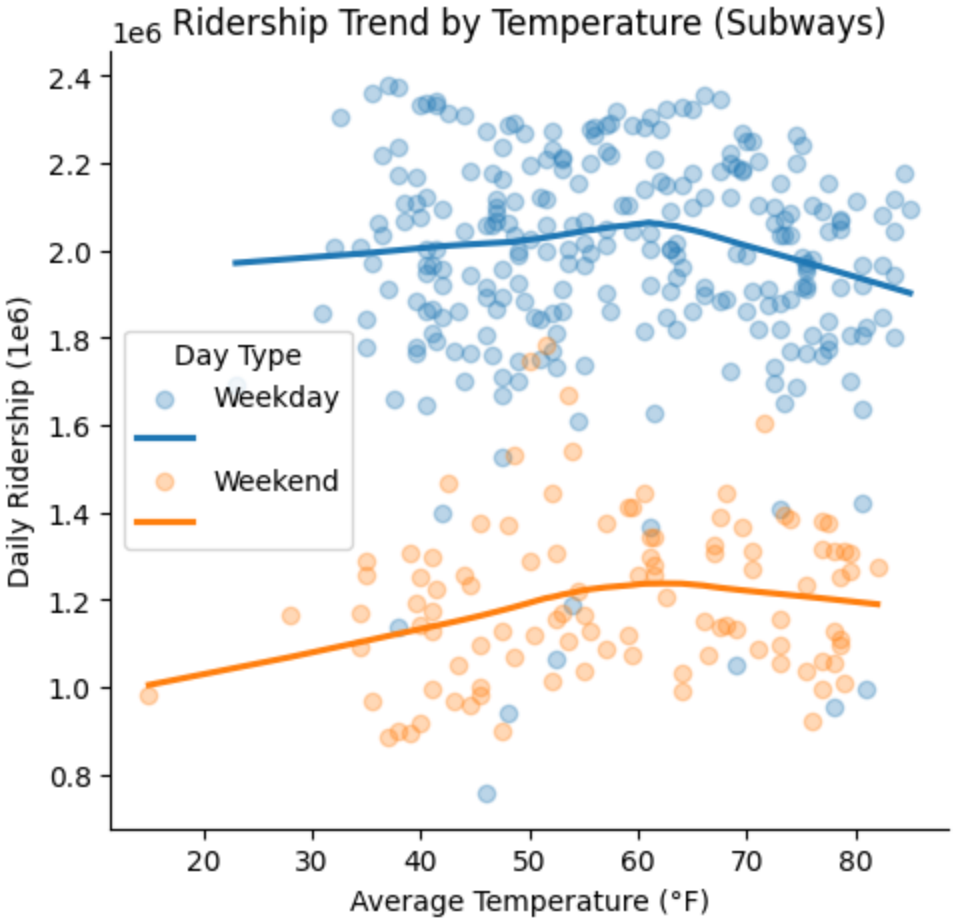
Overall, our lower threshold was the closest to the real peak and there are little visible relationships between average daily ride share trips, subway ridership, and daily temperature.

In [22]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

#import data
```

```
daily_weather = pd.read_csv("data/01-interim/Weather/CLEANED_CentralPark_Weather202
daily_ridership = pd.read_csv("data/01-interim/MTA_subway/MTA_Subway_Daily_Manhatta
daily_rides = pd.read_csv("data/02-processed/daily_total_ridership.csv")

daily_weather = daily_weather.drop(['Station','Name','Latitude','Longitude','Snow',
daily_weather['Date'] = pd.to_datetime(daily_weather['Date'])
daily_ridership['date'] = pd.to_datetime(daily_ridership['date'])
daily_rides['date'] = pd.to_datetime(daily_rides['date'])
```

In [23]:

```
#subway ridership and average temperature threshold (visual)
daily_ridership['is_weekend'] = daily_ridership['date'].dt.dayofweek > 4
merged = daily_ridership.merge(daily_weather, left_on='date', right_on='Date')
sns.lmplot(x='Avg_Temp', y='ridership', data=merged, hue='is_weekend', lowess = Tru
plt.legend(title = 'Day Type', labels = ['Weekday',' ','Weekend',' '], loc = 'cente
plt.title('Ridership Trend by Temperature (Subways)')
plt.xlabel('Average Temperature (°F)')
plt.ylabel('Daily Ridership (1e6)')
plt.subplots_adjust(right=0.8)
plt.tight_layout()
```




In [24]:

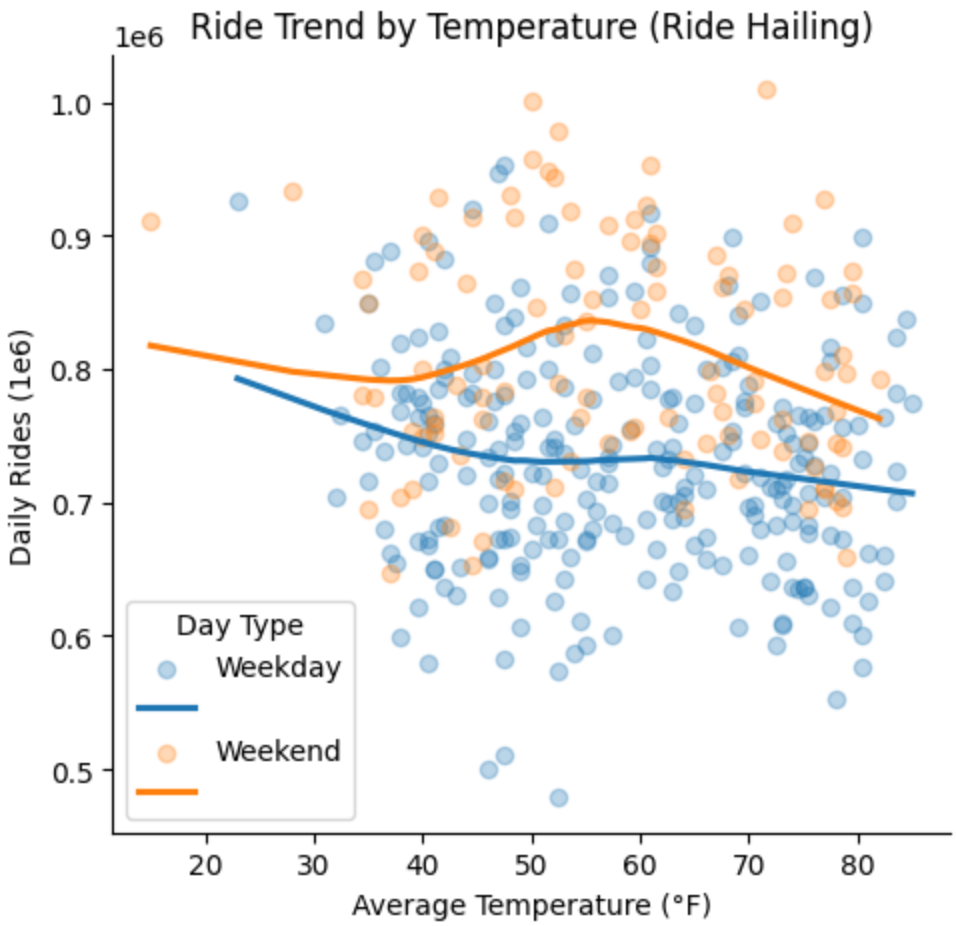
```
#subway ridership and average temperature threshold (numerical)
from statsmodels.nonparametric.smoothers_lowess import lowess

#define weekday/weekends
subway_weekdays = merged[~merged['is_weekend']].copy()
subway_weekends = merged[ merged['is_weekend']].copy()
```

```
#fit lowess
lowess_weekday = lowess(subway_weekdays['ridership'], subway_weekdays['Avg_Temp'])
lowess_weekend = lowess(subway_weekends['ridership'], subway_weekends['Avg_Temp'])

#find peaks
x_wd, y_wd = lowess_weekday.T
x_we, y_we = lowess_weekend.T

peak_temp_weekday = x_wd[np.argmax(y_wd)]
peak_ridership_wd = y_wd.max()

peak_temp_weekend = x_we[np.argmax(y_we)]
peak_ridership_we = y_we.max()

print(f"Weekday subway ridership peak at ~ {peak_temp_weekday:4.1f} °F, averaging {
print(f"Weekend subway ridership peak at ~ {peak_temp_weekend:4.1f} °F, averaging {
```

```
Weekday subway ridership peak at ~ 61.0 °F, averaging  2.06 million
Weekend subway ridership peak at ~ 62.5 °F, averaging  1.24 million
```

In [25]:

```
#ride hailing and average temperature threshold (visual)
daily_rides['date'] = pd.to_datetime(daily_rides['date'])

daily_rides['is_weekend'] = daily_rides['date'].dt.dayofweek > 4
merged2 = daily_rides.merge(daily_weather, left_on='date', right_on='Date')
sns.lmplot(x='Avg_Temp', y='total_daily_trips', data=merged2, hue='is_weekend', low
plt.legend(title = 'Day Type', labels = ['Weekday',' ','Weekend',' '], loc = 'lower
plt.title('Ride Trend by Temperature (Ride Hailing)')
plt.xlabel('Average Temperature (°F)')
plt.ylabel('Daily Rides (1e6)')
plt.subplots_adjust(right=0.8)
plt.tight_layout()
```




In [26]:
```
#ride hailing and average temperature threshold (numerical)
#define weekday/weekends
ridership_weekdays = merged2[~merged2['is_weekend']].copy()
ridership_weekends  = merged2[ merged2['is_weekend']].copy()

#filter ends
temp_min, temp_max = 45, 90

ridership_weekdays_filtered = ridership_weekdays[(ridership_weekdays['Avg_Temp'] >=
ridership_weekends_filtered = ridership_weekends[(ridership_weekends['Avg_Temp'] >=

#fit lowess
lowess_weekday2 = lowess(ridership_weekdays_filtered['total_daily_trips'], ridershi
lowess_weekend2 = lowess(ridership_weekends_filtered['total_daily_trips'], ridershi

#find peaks
x_wd2, y_wd2 = lowess_weekday2.T
x_we2, y_we2 = lowess_weekend2.T

peak_temp_weekday2 = x_wd2[np.argmax(y_wd2)]
peak_ridership_wd2 = y_wd2.max()

peak_temp_weekend2 = x_we2[np.argmax(y_we2)]
peak_ridership_we2= y_we2.max()

print(f"Weekday ride share rides peak at ~ {peak_temp_weekday2:4.1f} °F, averaging
print(f"Weekend ride share rides peak at ~ {peak_temp_weekend2:4.1f} °F, averaging
```

```
Weekday ride share rides peak at ~ 63.5 °F, averaging  0.74 million
Weekend ride share rides peak at ~ 57.0 °F, averaging  0.84 million
```

### ETC AD NASEUM

Some more words and stuff. Remember notebooks work best if you interleave the code that generates a result with properly annotate figures and text that puts these results into context.

In [27]:
```
## YOUR CODE HERE
## FEEL FREE TO ADD MULTIPLE CELLS PER SECTION
```

# Ethics

## A. Data Collection

- ☑ **A.1 Informed consent**: If there are human subjects, have they given informed consent, where subjects affirmatively opt-in and have a clear understanding of the data uses to which they consent?

> The datasets we are planning to use are directly from the public transit agencies. Due to the data being transit operations rather than individual user behavior, the traditional need for consent is not applicable. So I would say that the public transit riders may implicitly consent to data collection through the transit systems terms of use.

- ☑ **A.2 Collection bias**: Have we considered sources of bias that could be introduced during data collection and survey design and taken steps to mitigate those?

> Collection biases that we may come across are those who evade transportation fares as they would not be in the ridership counts. However, we believe this group to be negligible when compared to the overall ridership population. With this in mind, conclusions will always be estimations. Biases that would matter more include what time the data is taken, as the COVID era would be an outlier due to the changes during that year. Bus ridership during and out of school days would affect the ridership counts.

- ☑ **A.3 Limit PII exposure**: Have we considered ways to minimize exposure of personally identifiable information (PII) for example through anonymization or not collecting information that isn't relevant for analysis?

> The data set that we are using does not have any personally identifiable information. It consists of information such as aggregate ridership counts and schedule data.

- ☑ **A.4 Downstream bias mitigation**: Have we considered ways to enable testing downstream results for biased outcomes (e.g., collecting data on protected group status like race or gender)?

> The dataset does not include any demographic information such as race, gender, or socioeconomic status. Which would limit our ability to mitigate or even test for those factors, but also means that sensitive information is not at risk.

## B. Data Storage

- ☑ **B.1 Data security**: Do we have a plan to protect and secure data (e.g., encryption at rest and in transit, access controls on internal users and third parties, access logs, and up-to-date software)?

> The data is from public agencies and public accessible and contains no personally identifiable information, there is no need for more security measures beyond ethically using the data.

- ☐ **B.2 Right to be forgotten**: Do we have a mechanism through which an individual can request their personal information be removed?
- ☐ **B.3 Data retention plan**: Is there a schedule or plan to delete the data after it is no longer needed?

## C. Analysis

- ☐ **C.1 Missing perspectives**: Have we sought to address blindspots in the analysis through engagement with relevant stakeholders (e.g., checking assumptions and discussing implications with affected communities and subject matter experts)?

- ☑ **C.2 Dataset bias**: Have we examined the data for possible sources of bias and taken steps to mitigate or address these biases (e.g., stereotype perpetuation, confirmation bias, imbalanced classes, or omitted confounding variables)?

> We know that there is a risk of our sources being biased which would make our data biased. Socioeconomic factors could influence ridership patterns, as a more transit-dependent community may respond differently to frequency changes than less dependent riders. If fare data is involved, we can normalize based on the area's cost of living. Another factor to consider is the more heavily populated areas and high traffic places, may generate more ridership data. We will do our best to account for this by avoiding over generalizing any findings especially to those highly dense routes.

- ☐ **C.3 Honest representation**: Are our visualizations, summary statistics, and reports

designed to honestly represent the underlying data?

- ☑ **C.4 Privacy in analysis**: Have we ensured that data with PII are not used or displayed unless necessary for the analysis?

> Data with personal identifiable information was not collected, nor is it necessary for analysis, and thus will not be used or displayed.

- ☑ **C.5 Auditability**: Is the process of generating the analysis well documented and reproducible if we discover issues in the future?

> When generating the analysis, steps were taken to ensure the process is well documented and reproducible. These steps included keeping logs, frequent team communication, and transparency.

## D. Modeling

- ☑ **D.1 Proxy discrimination**: Have we ensured that the model does not rely on variables or proxies for variables that are unfairly discriminatory?

> Data used is sourced from multiple references, which helps reduce the possibility of discrimination, both directly and through proxy. However, there is concern for bias by income or socioeconomic class when looking at fares and the rider population.

- ☐ **D.2 Fairness across groups**: Have we tested model results for fairness with respect to different affected groups (e.g., tested for disparate error rates)?
- ☑ **D.3 Metric selection**: Have we considered the effects of optimizing for our defined metrics and considered additional metrics?

> Yes, optimized metrics such as fare costs compared to cost of living does not have any apparent unintended effects. Additional metrics can include the local cost of gas, local parking prices, and/or drivability of surrounding areas.

- ☐ **D.4 Explainability**: Can we explain in understandable terms a decision the model made in cases where a justification is needed?
- ☐ **D.5 Communicate limitations**: Have we communicated the shortcomings, limitations, and biases of the model to relevant stakeholders in ways that can be generally understood?

## E. Deployment

- ☐ **E.1 Monitoring and evaluation**: Do we have a clear plan to monitor the model and its impacts after it is deployed (e.g., performance monitoring, regular audit of sample predictions, human review of high-stakes decisions, reviewing downstream impacts of errors or low-confidence decisions, testing for concept drift)?

- ☑ **E.2 Redress**: Have we discussed with our organization a plan for response if users are harmed by the results (e.g., how does the data science team evaluate these cases and update analysis and models to prevent future harm)?

  > Our results are purely for informational purposes, not designed to be capable of harm. If harm through misrepresentation or misinterpretation were to arise, we can address the concerns on a case by case basis.

- ☑ **E.3 Roll back**: Is there a way to turn off or roll back the model in production if necessary?

  > Deletion or unpublication of our model can be easily done if necessary.

- ☑ **E.4 Unintended use**: Have we taken steps to identify and prevent unintended uses and abuse of the model and do we have a plan to monitor these once the model is deployed?

  > As our model is purely informational, the primary risk for unintended uses or abuse comes from possible misinterpretations and/or misapplications of our results (such as using our data to justify slowing transportation in lower income neighborhoods). Once deployed, we can monitor the use and citations of our analysis.

# Discusison and Conclusion

Wrap it all up here. Somewhere between 3 and 10 paragraphs roughly.

1. Summarize the important results you made
2. This is a good time to refer back to your Background section and review how this work extended the previous stuff.
3. Try to bring all the different results together into a bigger picture... what have we learned?
4. Often what we learned is confusing. Discuss if there are contradictory results how that might have happened
5. Discuss the limitations of your work. e.g, is the dataset a biased sample? would more data make you more confident? would longitudinal instead of cross-sectional data helpful? is there evidence that the phenomena you're investigating may change over time? etc
6. If you would like, you can also say what the next steps in this investigation might be if the work was to continue outside this class